



UNIVERSIDADE FEDERAL DE UBERLÂNDIA
FACULDADE DE ENGENHARIA ELÉTRICA
GRADUAÇÃO EM ENGENHARIA ELETRÔNICA E DE TELECOMUNICAÇÕES

Igor Braga Santos

Implementação de uma Rede Definida por Software com princípios de Zero Trust Network Access

Patos de Minas - MG
2024

Igor Braga Santos

Implementação de uma Rede definida por software com princípios de Zero Trust Network Access

Projeto Final de Curso apresentado à banca examinadora como requisito parcial de avaliação da disciplina de PFC2 da graduação em Engenharia Eletrônica e de Telecomunicações, da Faculdade de Engenharia Elétrica, da Universidade Federal de Uberlândia, Campus Patos de Minas.

Orientador: Prof. Dr. Rafael Augusto da Silva

Patos de Minas - MG

2024

Igor Braga Santos

Implementação de uma Rede definida por software com princípios de Zero Trust Network Access

Projeto Final de Curso apresentado à banca examinadora como requisito parcial de avaliação da disciplina de PFC2 da graduação em Engenharia Eletrônica e de Telecomunicações, da Faculdade de Engenharia Elétrica, da Universidade Federal de Uberlândia, Campus Patos de Minas.

Orientador: Prof. Dr. Rafael Augusto da Silva

Banca Examinadora:

Prof. Dr. Rafael Augusto da Silva
FEELT/UFU (Orientador)

Prof. Dr. Laurence Rodrigues do Amaral
FACOM/UFU (Membro 1)

Prof.^a Dr.^a Karine Barbosa Carbonaro
FEELT/UFU (Membro 2)

Agradecimentos

Inicialmente gostaria de agradecer a Deus, que me guiou com saúde e força para superar todos os objetivos e obstáculos que encontrei durante a graduação.

Agradeço ao meu pai Valdecássio Ribeiro, que me amparou, ensinou e provou durante anos que você pode chegar onde quiser independente de suas dificuldades e origens. Agradeço à minha mãe, Lélia Braga, pelas orações e por todo o processo que me edificou para me tornar um homem apresentável. Agradeço também às minhas irmãs.

Agradeço à minha esposa Victoria Ribeiro por todas as vezes que me lembrou que sou capaz, por todos os dias difíceis que ali eu teria um porto seguro, por todos os momentos de felicidade proporcionados durante o processo e por todas as oportunidades que se abriram para mim.

Agradeço ao José Eduardo e a Elizabeth Ribeiro, que me acolheram como um filho e me colocaram em posições de oportunidade e crescimento de conexões profissionais.

Agradeço à todos os professores da Universidade Federal de Uberlândia que me encorajaram e me auxiliaram durante o curso e o desenvolvimento deste trabalho.

E finalmente, agradeço ao meu orientador Rafael Augusto que investiu altruísmo, interesse, paciência, conhecimento e esforço para elaborar este projeto.

*“Podem tirar minha casa, todos os meus truques e brinquedos, mas uma coisa não
podem tirar: eu sou o Homem de Ferro!”
(Anthony Edward Stark, Homem de Ferro 3)*

Resumo

A crescente complexidade das redes e a necessidade de segurança em ambientes distribuídos impulsionam a adoção de arquiteturas que sejam capazes de simplificar o processo, que permitam o controle centralizado e a flexibilidade na configuração de políticas de rede. A *Software Defined Network* (SDN) tem o intuito de transformar a infraestrutura física de redes de computadores convencionais em um ambiente virtualizado modelável e escalável. Paralelamente o *Zero Trust Network Access* (ZTNA) reforça a segurança, aplicando o conceito de "confiança zero" onde todos os dispositivos e fluxos de dados passam por autenticação contínua e segmentação rigorosa. Aliado a essas tecnologias, o uso de contêineres, principalmente ferramentas como *Docker* e *Docker Compose*, possibilitam uma implementação simplificada, replicável e escalável de ambientes virtualizados. O presente trabalho propõe o projeto e a implementação de uma rede definida por software fundamentada nos princípios de segurança da arquitetura ZTNA. O objetivo central é desenvolver um ambiente de rede que seja escalável, robusto e seguro, capaz de se adaptar tanto a projetos simples de baixo custo quanto a propostas mais complexas e de maior escala. A arquitetura *Zero Trust* é escolhida como base devido à sua abordagem proativa e centrada na segurança, que desafia a tradicional confiança implícita em redes corporativas. Ao implementar os princípios do *Zero Trust* o sistema visa garantir que todos os dispositivos, usuários e aplicativos sejam continuamente autenticados e autorizados. A abordagem SDN é adotada para proporcionar flexibilidade e agilidade à rede, permitindo a configuração dinâmica e programática dos recursos de rede.

Palavras-chave: Redes de Computadores; SDN; Segurança de Redes; Contêineres; ZTNA.

Abstract

The growing complexity of networks and the need for security in distributed environments have driven the adoption of architectures that are capable of simplifying the process, allowing centralized control and flexibility in configuring network policies. The SDN aims to transform the physical infrastructure of conventional computer networks into a modelable and scalable virtualized environment. At the same time, ZTNA reinforces security by applying the concept of "zero trust" where all devices and data flows undergo continuous authentication and rigorous segmentation. Combined with these technologies, the use of containers, especially tools such as *Docker* and *Docker Compose*, enable a simplified, replicable and scalable implementation of virtualized environments. This work proposes the design and implementation of a software-defined network based on the security principles of the ZTNA architecture. The central objective is to develop a network environment that is scalable, robust and secure, capable of adapting to both simple low-cost projects and more complex and large-scale proposals. The *Zero Trust* architecture is chosen as the basis due to its proactive and security-centric approach, which challenges the traditional implicit trust in corporate networks. By implementing the *Zero Trust* principles, the system aims to ensure that all devices, users and applications are continuously authenticated and authorized. The SDN approach is adopted to provide flexibility and agility to the network, allowing dynamic and programmatic configuration of network resources.

Keywords: Computer Networks; SDN; Network Security; Containers; ZTNA.

Lista de ilustrações

Figura 1.1 – Crescimento do mercado de orquestração SDN.	15
Figura 1.2 – Ciclo de cronologia e impressões do ZTNA até 2023.	16
Figura 1.3 – Uso inadequado de dispositivos no ambiente de trabalho.	17
Figura 2.1 – Arquitetura de uma SDN.	20
Figura 2.2 – Exemplo de entrada em uma tabela de fluxo compatível com <i>OpenFlow</i>	22
Figura 2.3 – Modelo lógico com principais componentes de um modelo <i>Zero Trust</i>	23
Figura 2.4 – Organização de um ataque DDoS.	26
Figura 3.1 – Políticas de segurança da rede.	30
Figura 3.2 – Fluxo de uma tentativa de conexão a rede.	31
Figura 3.3 – Listagem e informações dos contêineres.	40
Figura 4.1 – Resultado da execução do arquivo "l2.py".	42
Figura 4.2 – Topologia criada pelo <i>script</i> do <i>Mininet</i>	43
Figura 4.3 – Resultado do ping do primeiro arquivo no terminal do <i>Ryu</i>	44
Figura 4.4 – Resultado do ping do primeiro arquivo no terminal do <i>Mininet</i>	44
Figura 4.5 – Capacidade de transferência de dados do protocolo <i>Transmission Control Protocol</i> (TCP) para o arquivo "l2.py".	45
Figura 4.6 – Capacidade de transferência de dados do protocolo <i>User Datagram Protocol</i> (UDP) para o arquivo "l2.py".	45
Figura 4.7 – <i>Logs</i> da abertura do arquivo.	46
Figura 4.8 – <i>Logs</i> do carregamento da topologia	46
Figura 4.9 – <i>Logs</i> do <i>Ryu</i>	47
Figura 4.10–Teste de h1 para h2.	47
Figura 4.11–Teste de h2 para h1.	47
Figura 4.12–Teste de h1 para h3.	48
Figura 4.13–Teste de h3 para h1.	48
Figura 4.14–Teste de h2 para h3.	49

Figura 4.15–Teste de h3 para h2.	49
Figura 4.16–Capacidade de transferência de dados com o protocolo TCP com as regras de segurança.	50
Figura 4.17–Capacidade de transferência de dados com o protocolo UDP com as regras de segurança.	51

Lista de Códigos

3.1	Instalação do docker e Docker Compose	33
3.2	Criação e abertura do diretório	33
3.3	Definições do Docker Compose	33
3.4	Definições do arquivo de comunicação simples	34
3.5	Definições do arquivo com regras de segurança	35
3.6	Topologia utilizada em código	38
3.7	Código adaptado para o switch OpenFlow 1.3	39
3.8	Execução da aplicação	40
3.9	Verificação dos contêineres	40
3.10	Execução do terminal de interação do Ryu	40
3.11	Execução do terminal de interação do Mininet	41
3.12	Transferência dos arquivos para os contêineres	41
4.1	Execução do arquivo de comunicação	42
4.2	Execução da topologia	42
4.3	Envio de pacotes de teste	43
4.4	Execução do arquivo com regras de segurança	46
4.5	Execução da topologia com switch OpenFlow	46
4.6	Cálculo de banda entre h1 e h2 no TCP	50
4.7	Cálculo de banda entre h1 e h2 no UDP	50

Lista de siglas

API *Application Programming Interfaces*

CPU *Central Processing Unit*

DDoS *Distributed Denial of Service*

DoS *Denial of Service*

DNS *Domain Name System*

FIB *Forwarding Information Base*

HTTP *Hypertext Transfer Protocol*

IZTSDN *Intelligent Zero Trust Software Defined Networking*

IIoT *Industrial Internet of Things*

ICMP *Internet Control Message Protocol*

LAN *Local Area Network*

NBI *Northbound Interface*

NOS *Network Operating System*

PDP *Policy Decision Point*

PE *Policy Engine*

PA *Policy Administrator*

PEP *Policy Enforcement Point*

RIB *Routing Information Base*

RSA *Rivest Shamir Adleman*

SDN *Software Defined Network*

TCP *Transmission Control Protocol*

UDP *User Datagram Protocol*

ZTNA *Zero Trust Network Access*

ZTA *Zero Trust Architectures*

Sumário

1	INTRODUÇÃO	14
1.1	Problematização	16
1.2	Objetivos	17
1.2.1	Objetivo Geral	17
1.2.2	Objetivos Específicos	17
1.3	Justificativas	18
2	FUNDAMENTAÇÃO TEÓRICA	19
2.1	Arquitetura de Redes Definidas por Software	19
2.1.1	Plano de Gerenciamento	20
2.1.2	<i>Northbound Interface</i>	21
2.1.3	Plano de Controle	21
2.1.4	<i>Southbound Interface</i>	21
2.1.5	Plano de Dados	22
2.2	Arquitetura <i>Zero Trust Network Access</i>	23
2.2.1	PDP	24
2.2.2	PEP	24
2.3	Containerização	24
2.4	Tipos mais comuns de ameaças a redes de computadores	25
2.4.1	<i>Phishing</i>	25
2.4.2	Negação de Serviço Distribuído	25
2.4.3	<i>Ransomware</i>	26
2.5	Estado da Arte	27
3	MATERIAIS E MÉTODOS	29
3.1	Mininet	31
3.2	<i>Ryu</i>	31
3.3	<i>OpenFlow</i>	32

3.4	<i>Docker</i>	32
3.4.1	<i>Docker Compose</i>	32
3.5	Instalação e configuração do <i>Docker Compose</i>	33
3.6	Configuração e importação dos arquivos <i>Python</i> para o <i>Ryu</i> . .	34
3.7	Configuração do <i>Mininet</i>	37
3.8	Execução dos contêineres	40
3.9	Movendo os arquivos <i>Python</i> criados para os respectivos con- têineres	40
4	RESULTADOS	42
4.1	Abertura do arquivo "l2.py" e topologia do <i>Mininet</i>	42
4.2	Testando a conectividade da topologia	43
4.3	Capacidade de transferência de dados dos protocolos TCP e UDP	45
4.4	Teste das regras de segurança	45
4.5	Resultados de banda para protocolo TCP e UDP com as polí- ticas de segurança	49
5	CONSIDERAÇÕES FINAIS	52
	REFERÊNCIAS	53

Introdução

Desde o início da história da tecnologia da informação, o ramo passou por constante evolução quando se engloba armazenamento, processamento, rede e periféricos. Primeiramente, os dispositivos operavam de forma separada, cada um com seu padrão físico (Nadeau; Gray, 2013).

Na década de 1990, os dispositivos de armazenamento, processamento, rede e periféricos seguiam um padrão físico, ou seja, cada tipo de dispositivo, como armazenamento, processamento pelo *Central Processing Unit* (CPU), rede (*switches* e roteadores) e periféricos (impressoras, *scanners*), era fisicamente separado, com *hardware* e *software* específicos e funções rígidas. Esses dispositivos operavam de maneira independente e seguiam protocolos e interfaces fixas, muitas vezes proprietárias, que exigiam configurações manuais para interoperabilidade. Quando houve uma demanda maior por principalmente, processamento, foi observado uma necessidade de unir todos esses elementos visando principalmente, a redução de custos (Nadeau; Gray, 2013).

As bordas de rede, cada vez mais, têm um crescimento gradativo de dispositivos (aplicações e hospedeiros). Este aumento gera uma demanda maior de dispositivos no núcleo (roteadores e redes de redes) (Nadeau; Gray, 2013). A partir do momento em que um sistema passa a escalar deste modo, vários fatores se tornam complexos de serem trabalhados.

O grande porte de redes de computadores convencionais remetem em menores níveis de segurança. Isso, pois, usuários finais como celulares, computadores ou estações de trabalho estão sujeitos a um maior risco de ataques e servem como conectivos para ataques a um ambiente mais preciso (Kurose; Ross, 2013). O ataque do tipo Negação de Serviço - *Denial of Service* (DoS), por exemplo, é um tipo de ameaça que pode ser evitada com visibilidade e controle centralizado.

A falta de implementação de políticas de segurança, ou seja, as regras e limitações que podem determinar o limite de iteração e acesso da rede e muitas vezes a falta de treinamento de segurança de dados para funcionários, podem qualificar uma ameaça à empresa de ataques cibernéticos (Nadeau; Gray, 2013; Kurose; Ross, 2013).

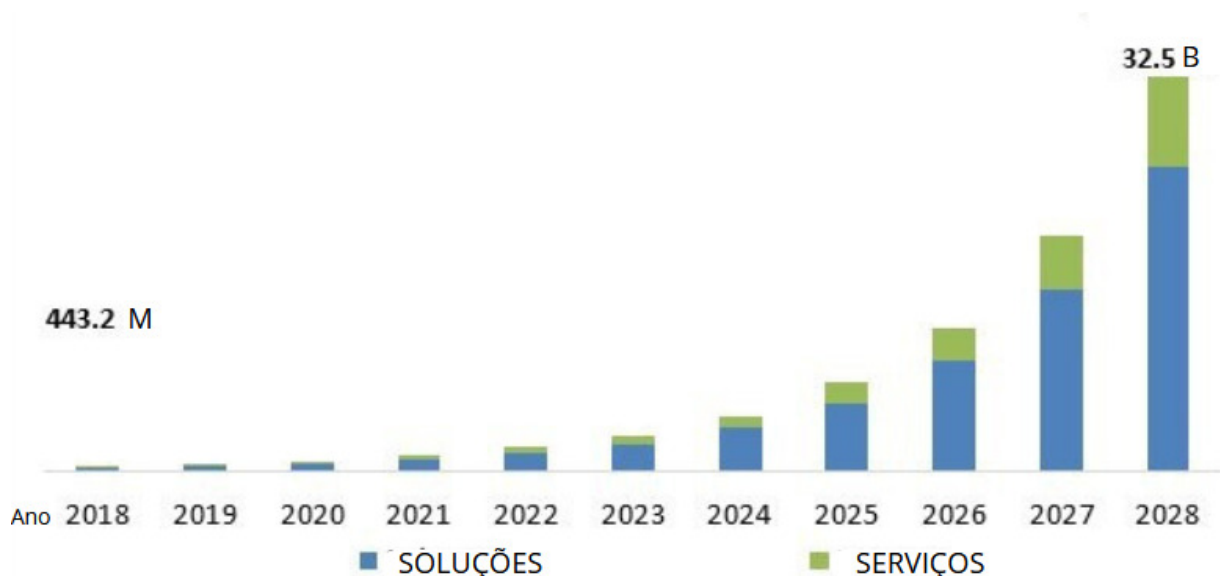
Arquiteturas de rede que não foram planejadas para ter escalabilidade enfrentam grande dificuldade para ter eficiência em seu crescimento, pois muitas vezes, existe incompatibilidade da estrutura ou dos recursos existentes em um sistema. Expansões de rede podem requerer adequação de um cenário sobre equipamentos e uma mão de obra elevada (Kurose; Ross, 2013).

Os ambientes de computação em nuvem, onde o usuário é hospedado em um servidor remoto, traz uma possibilidade de utilização de serviços como gerenciamento de dados e poder de computação estão se tornando muito eficientes para empresas que não desejam ser o gerenciador ativo das aplicações (Kurose; Ross, 2013).

Ao adquirir um roteador ou um *switch* comercial, o público se depara com dispositivos que possuem interfaces configuráveis por um profissional qualificado em redes (Nadeau; Gray, 2013). Essas interfaces, apesar de possuírem algumas ferramentas que podem fazer com que a rede seja mais segura, são limitadas, pois são interfaces de código fonte fechado.

A SDN é uma arquitetura de rede que dispensa dispositivos com funções específicas, como um *firewall* ou um *switch* padrão. Ao invés disso, essas redes utilizam software como sua maneira de criar e administrar serviços de redes (Atlântico, 2017). Neste cenário, o plano de controle e o plano de encaminhamento são separados, tornando o plano de controle programável, abstendo de infraestruturas subjacentes (Stefanini, 2018). A Figura 1.1 mostra um gráfico de como o mercado de soluções e serviços de SDN tem se comportado e quais são as expectativas e previsões para os anos em sequência até 2028.

Figura 1.1 – Crescimento do mercado de orquestração SDN.

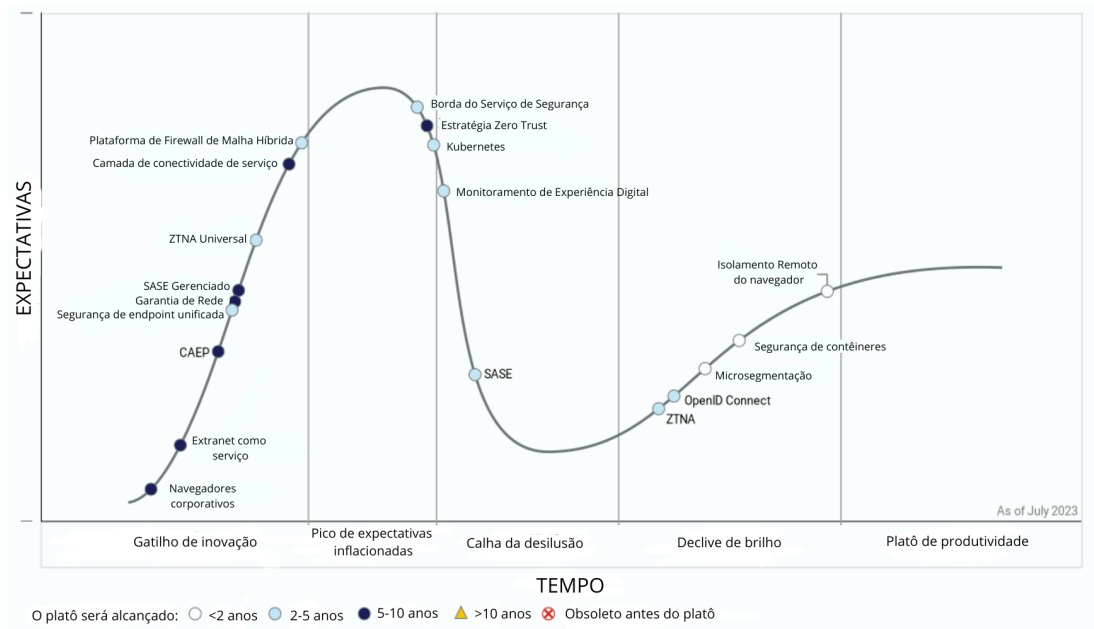


Fonte: Adaptado de KBV (2022, on-line)

O ZTNA trata-se de um padrão de arquitetura de rede onde cada usuário, seja ele local ou remoto, é sujeito a uma verificação de segurança dinâmica. Autenticação, autorização e validação são etapas no qual o modelo é construído. As regras de acesso são configuráveis

de acordo com a necessidade do usuário. Seu lema é: nunca confiar, sempre verificar (PingIdentity, 2021). Um *insight* de como o ZTNA se comportou em alguns anos é ilustrado na Figura 1.2.

Figura 1.2 – Ciclo de cronologia e impressões do ZTNA até 2023.



Fonte: Adaptado de Lerner e Watts (2023, on-line)

1.1 Problematização

Segundo o *DevPro Journal* o mercado de SDN, que foi avaliado em 9 bilhões de dólares em 2019, tem expectativa de crescimento de 28%, ou seja, 72,63 bilhões de dólares (Majewski, 2023). Com a expansão deste mercado fica muito explícito a importância dos investimentos e pesquisas nas redes definidas por *software*.

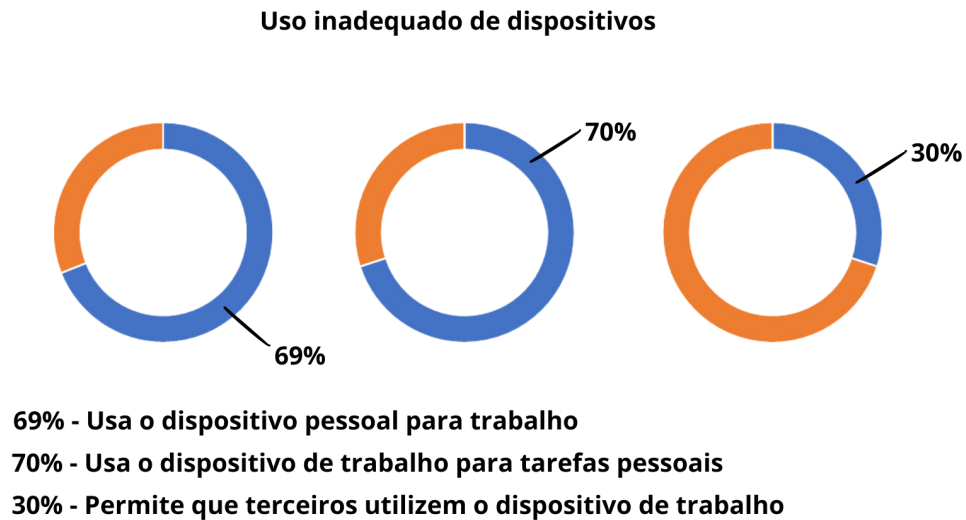
A grande possibilidade de acesso a diferentes serviços na internet atualmente, além de oferecer diversos parâmetros que podem facilitar as tarefas do usuário, também o torna exposto. A partir disso, é muito importante o cuidado com os cliques e redirecionamento de páginas que podem comprometer os dados e o sistema de uma rede.

De acordo com o *Alliance Virtual Offices*, houve o aumento extremo do trabalho remoto provocado pelo Covid-19 durante a pandemia. Essa modalidade de ambiente abre portas para ameaças às empresas e os funcionários. O relatório mostra que os ataques cibernéticos aumentaram 238% na pandemia (Blandford, 2023).

O uso dos dispositivos de trabalho para uso pessoal é um dos principais motivos que intensificam ataques à segurança. Os números vão além quando funcionários utilizam seu dispositivo pessoal para trabalho ou emprestam o mesmo a outras pessoas (Blandford,

2023). As porcentagens dos cenários inadequados de dispositivos podem ser vistas na Figura 1.3.

Figura 1.3 – Uso inadequado de dispositivos no ambiente de trabalho.



Fonte: Adaptado de Blandford (2023, on-line)

1.2 Objetivos

1.2.1 Objetivo Geral

O trabalho em questão tem como objetivo desenvolver um ambiente de rede virtualizado aplicando princípios de segurança para maior gerenciamento centralizado, grande capacidade de escalabilidade e redução de custos. Essa tecnologia combinada com os princípios da arquitetura ZTNA deve apresentar maior segurança de tráfego e controle granular de atividades específicas a cada tipo de usuário.

1.2.2 Objetivos Específicos

- Implementar uma SDN simulando um ambiente empresarial ou doméstico;
- Implantar políticas de segurança baseadas nos princípios de ZTNA;
- Avaliar o desempenho do sistema através de métricas de operação e segurança.

1.3 Justificativas

O crescimento de dispositivos na borda da rede que estão descentralizados e a exposição dos mesmos à internet, requerem a adoção de novas tecnologias. O uso de uma SDN ajustada com regras de segurança baseadas na arquitetura ZTNA se justificam com algumas vantagens, como:

- Monitoramento e controle de encaminhamento;
- Sistema centralizado e escalável;
- Redução de custos com dispositivos tradicionais;
- Controle granular de usuários;
- Segurança de redes e sistemas.

Os pontos apresentados oferecem um breve resumo dos inúmeros benefícios que um projeto com esta estrutura pode proporcionar. Além disso, a área de segurança e redes pode contribuir significativamente para o crescimento econômico do país, gerando um volume de capital ainda maior.

A escalabilidade que um sistema virtualizado pode oferecer facilmente cobre a substituição de equipamentos extremamente utilizados por empresas de grande porte, que possuem custos elevados. Além disso, a utilização inteligente de um processador, principalmente em redes de grande porte, torna-se uma otimização indispensável.

Com a SDN e os princípios do modelo ZTNA trabalhando em conjunto, as organizações podem implementar políticas de segurança dinâmicas e adaptáveis que respondem rapidamente as ameaças comuns e emergentes. Isso é especialmente importante em um cenário de segurança cibernética em constante evolução, onde novas ameaças estão sempre surgindo.

Fundamentação Teórica

2.1 Arquitetura de Redes Definidas por Software

O conceito de uma Rede Definida por Software remete a uma arquitetura de rede que separa o plano de controle (responsável por decisões de encaminhamento) do plano de dados (responsável pelo encaminhamento real dos pacotes). Essa arquitetura permite que o controle da rede seja programável e centralizado em um controlador SDN, facilitando a gestão e a automação da rede, além de tornar sua infraestrutura mais flexível e ágil (Nadeau; Gray, 2013).

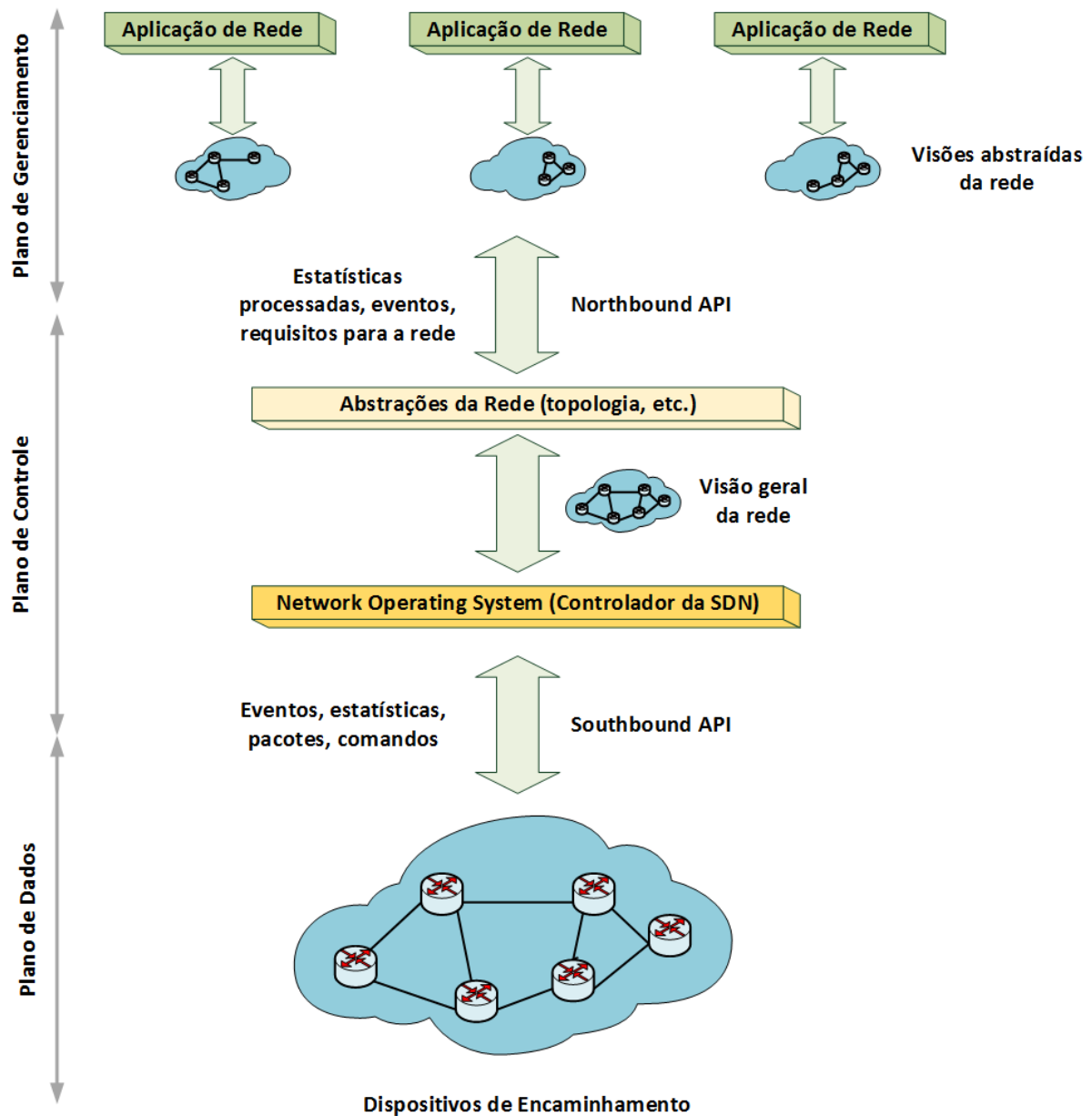
Uma SDN permite que os administradores configurem e controlem a rede por meio de software, em vez de depender exclusivamente de dispositivos de rede físicos e suas configurações individuais. Esse conceito visa superar as limitações das redes tradicionais, que têm o hardware e o software fortemente acoplados, dificultando mudanças e inovações rápidas. O modelo desta arquitetura facilita a adaptação da rede a novas requisições, possibilita uma melhor utilização dos recursos e proporciona uma visão unificada e centralizada de toda a topologia de rede.

No quesito de arquitetura destas redes, é possível efetuar uma divisão em cinco seções principais, com suas respectivas funções (Gomes et al., 2015). São elas:

- Plano de gerenciamento;
- Plano de controle;
- Plano de dados;
- *Northbound Interface*;
- *Southbound Interface*.

Referente a estas partes citadas, é possível ter maior agnição segundo a Figura 2.1.

Figura 2.1 – Arquitetura de uma SDN.



Fonte: Gomes et al. (2015, on-line)

2.1.1 Plano de Gerenciamento

O plano que está entre a rede e o usuário em questão que vai administrar a mesma é o plano de gerenciamento. A partir desta seção serão feitas as definições de funções da rede, a coleta de eventos, a verificação do status, entre outros (Gomes et al., 2015).

A linguagem de programação aliada a uma *Application Programming Interfaces* (API), fará com que o plano de gerenciamento e o controlador tenham comunicação que faça o sistema funcionar. As APIs se encontram em uma camada intermediária entre aplicações para garantir essa ligação. Ao fazer uma requisição para uma API, haverá uma consulta no banco de dados de um servidor que ela utiliza que trará a informação de volta para o

usuário (Johl, 2022).

Na arquitetura de rede utilizada neste trabalho pretende-se utilizar duas APIs: a *Northbound Interface* e a *Southbound Interface*.

2.1.2 *Northbound Interface*

Quando existe a necessidade de uma comunicação entre dois componentes de rede que se expressam em linguagens diferentes (nível superior e inferior), é essencial o uso de uma API. A *Northbound Interface* (NBI) será a interface que fará a conexão do alto nível de requisitos técnicos e o sistema que irá administrar a rede (Tuncer et al., 2018).

Em outras palavras, a NBI é responsável pela comunicação entre o plano de gerenciamento e o plano de controle em um ambiente de SDN. Seu uso possibilita que o administrador com suas aplicações seja capaz de gerenciar as funções da rede sem se preocupar com os comandos de nós da rede (Gomes et al., 2015).

2.1.3 Plano de Controle

De uma maneira geral, como o nome sugere, o plano de controle é definido pelo bloco de um ou mais controladores operantes em *software* que visam comandar e supervisionar o encaminhamento de rede (Karakus; Durresi, 2016). A partir de regras específicas criadas pelo administrador de rede, o controlador é a seção da SDN que processa os dados e o encaminhamento de acordo com o que foi programado.

Aqui será concentrado o engenho da SDN. Diante de um *Network Operating System* (NOS) é possível monitorar as informações de funcionamento da rede, a organização dos dispositivos conectados, encaminhamento de pacotes, possibilidade de implementações de segurança e gerenciamento dos dados transmitidos nessa rede (Gomes et al., 2015).

2.1.4 *Southbound Interface*

A interface chamada de *Southbound Interface* também se trata de uma API que desta vez efetuará a comunicação entre o plano de controle e o plano de dados. A API localizada no sul da arquitetura de uma SDN é capaz de disponibilizar um controle maleável do encaminhamento, avisos de capacidade, apresentação de resultados e alerta de eventos (Prajapati; Sakadasariya; Patel, 2018).

A interface mais utilizada na *Southbound Interface* é o *OpenFlow*, que será esclarecido em capítulos seguintes. Por outro lado, existem diversas APIs que podem efetuar um trabalho semelhante como OVSD, ForCES e OpFlex. Cada uma destas soluções tem um recurso que pode ou não ser mais conveniente quando comparadas umas com as outras (Gomes et al., 2015).

2.1.5 Plano de Dados

O plano de dados vai se responsabilizar por resolver os datagramas que ele receberá baseado em operações no enlace que são coletadas e verificadas. Este mesmo datagrama será processado por uma ou mais tabelas *Forwarding Information Base* (FIB) através do plano de dados se estiver bem definido. Caso não esteja, este mesmo datagrama pode ser encaminhado e processado pela tabela *Routing Information Base* (RIB) através do plano de controle (Nadeau; Gray, 2013).

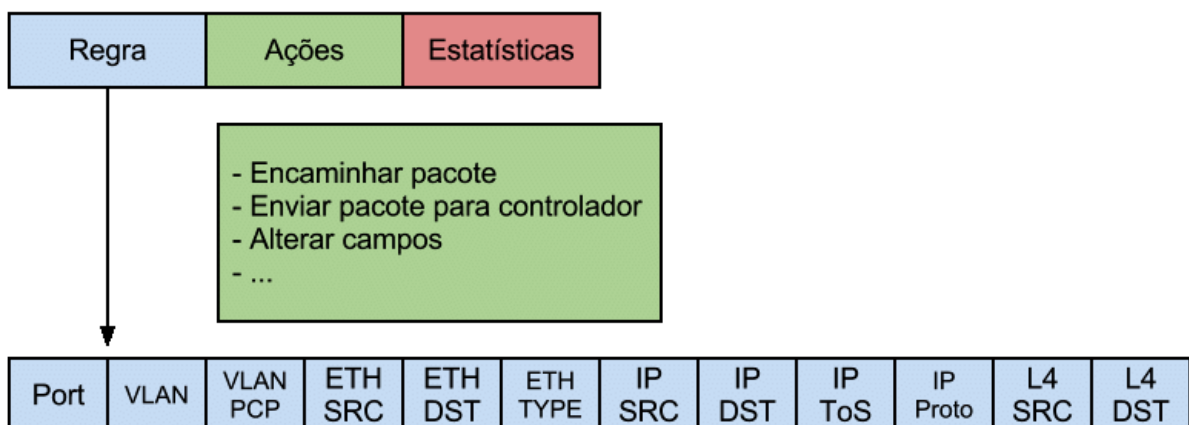
A tabela FIB geralmente está presente nos dispositivos de encaminhamento como por exemplo, os *switches* compatíveis com o protocolo *OpenFlow*. Seu objetivo e principal diferença é tomar decisões mais rápidas de encaminhamento de pacotes para seu destino (Yan et al., 2015).

Essas tabelas, também chamadas de tabelas de fluxo, possuem regras de compatibilidade. Estas regras consistem em uma validação de valores de campos do cabeçalho do conjunto de dados do pacote bit por bit. O pacote pode ser submetido a diferentes ações caso passe na verificação das regras (Gomes et al., 2015). Neste trabalho as três ações que mais serão exploradas são:

- Encaminhamento do pacote ao destino solicitado;
- Relatório do pacote encapsulado para o controlador;
- Descarte do pacote.

Um exemplo de uma tabela de fluxo é representado pela Figura 2.2.

Figura 2.2 – Exemplo de entrada em uma tabela de fluxo compatível com *OpenFlow*.



Fonte: Guedes et al. (2014, on-line)

2.1.5.1 Dispositivos de Encaminhamento

Diferente das redes tradicionais, os dispositivos de encaminhamento são desacoplados do plano de controle e em uma SDN serão responsáveis por encaminhar os pacotes a

determinadas portas no plano de dados. O gerenciamento destes dispositivos será feito pelo controlador no plano de controle (Nadeau; Gray, 2013).

Os dispositivos de encaminhamento geralmente utilizados atualmente no mercado para orquestração de uma SDN são os *switches* gerenciáveis compatíveis com o protocolo *Open-Flow*. Por outro lado, também é possível observar roteadores ou implementações de firewalls compatíveis na estrutura de uma rede definida por software.

Existe a possibilidade de efetuar a virtualização destes dispositivos de encaminhamento, o que será trabalhado em capítulos seguintes.

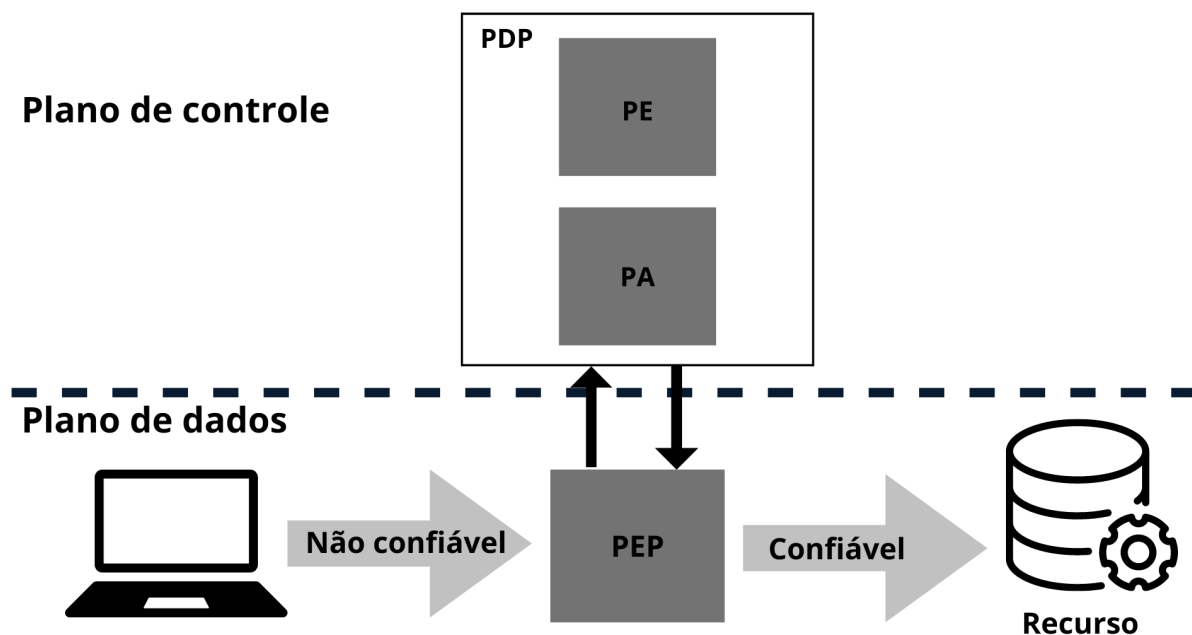
2.2 Arquitetura *Zero Trust Network Access*

A arquitetura de uma rede baseada em ZTNA é capaz de cobrir os pontos de falhas de segurança que uma rede tradicional não cobre. Partindo do ponto que todos os dispositivos, sejam eles de uma rede interna ou externa, não são confiáveis, o acesso granular pode ser altamente escalável e modelável.

Ao efetuar uma tentativa de acesso a uma rede com os princípios de ZTNA, o dispositivo passa por uma série de verificações e de acordo com estas verificações e testes, pode receber diferentes níveis de acesso em ambientes da rede (Gilman; Barth, 2017).

A Figura 2.3 supõe através de um modelo lógico que qualquer sistema, dispositivo, usuário ou aplicação faça um requerimento de um recurso na rede.

Figura 2.3 – Modelo lógico com principais componentes de um modelo *Zero Trust*.



Fonte: Adaptado de Srinivasan (2023, on-line)

O modelo é dividido em dois planos: o plano de controle e o plano de dados. O plano

de controle engloba o *Policy Decision Point* (PDP) e o plano de dados engloba o *Policy Enforcement Point* (PEP).

2.2.1 PDP

Como pode ser visto na Figura 2.3, o PDP é um ponto de decisão que pode ser dividido em dois pontos de decisão: o *Policy Engine* (PE) e o *Policy Administrator* (PA).

Ao receber uma solicitação de acesso, o PE irá decidir se o usuário está apto ou não de acordo com as regras impostas e a informações externas. O *Policy Engine* envia os dados para um algoritmo de confiança pré definido que retorna para o executor se o acesso deve ser autorizado, negado ou revogado (Gilman; Barth, 2017).

O PA é encarregado de estabelecer ou encerrar uma conexão entre o solicitador e o recurso solicitado. Este administrador e o PE agem em conjunto no PDP (Srinivasan, 2023).

2.2.2 PEP

Localizado no plano de dados, o PEP irá intermediar a solicitação (usuário, dispositivo, aplicação, etc.) e o recurso solicitado. Nesta seção há a coleta e o encaminhamento dos dados fornecidos pelo sujeito, e assim, com a tomada de decisão dos componentes do PDP a conexão será estabelecida, monitorada, encerrada ou revogada (Alcaraz; Lopez; Wolthusen, 2016; Srinivasan, 2023).

2.3 Containerização

Contêineres fazem parte de uma tecnologia que permite enfiar e isolar uma aplicação e seus requerimentos em um ambiente só, garantindo que ela seja executada de maneira consistente em diferentes infraestruturas (Siddiqui; Siddiqui; Khan, 2019).

Por compartilharem o sistema operacional e bibliotecas do *host* que os executa, os contêineres levam vantagem quando comparados a máquinas virtuais, pois, serão encapsulados apenas os arquivos e configurações necessárias para que uma aplicação seja executada. Portanto, tudo o que não é necessário de um sistema operacional ou outras aplicações não fará parte da composição de uma determinada aplicação em um contêiner (Singh; Peddoju, 2017).

2.4 Tipos mais comuns de ameaças a redes de computadores

No ambiente das máquinas, usuários lidam com a internet e *softwares* o tempo inteiro, ao estar operando ou conectado. Segundo Bruce Schneier, “o único computador seguro é aquele que está desligado, trancado em um cofre e enterrado a 6 metros de profundidade em um local secreto – e também não estou totalmente confiante quanto a isso” (Osanaïye; Choo; Dlodlo, 2016).

Como citado no capítulo anterior, os dispositivos que são disponibilizados para atividades trabalhistas acabam ficando sujeitos ao uso pessoal ou mesmo de terceiros. Essa prática, em controle de quem não possui um devido treinamento de segurança torna o ambiente de trabalho extremamente exposto à ataques.

2.4.1 *Phishing*

O cenário onde a engenharia social é utilizada para encontrar fraquezas nos usuários de uma rede ou de um sistema e roubar informações confidenciais de um ambiente é chamado de *phishing* (Khonji; Iraqi; Jones, 2013).

As informações coletadas pelo *phishing* geralmente são dados pessoais de um usuário, como nome, senhas, documentos, dados de um cartão de crédito ou até mesmo informações familiares. O grande problema no combate deste ataque se dá pela vulnerabilidade humana, uma vez que o problema de segurança não se encontra na máquina ou no sistema (Aleroud; Zhou, 2017).

Os meios de ataque do *phishing* são situações que se passam por um ambiente usual ou um hábito local. O *hacker* por exemplo, pode simular um *link Hypertext Transfer Protocol* (HTTP) solicitando novas credenciais de usuário e para informar as atuais. Outro exemplo seria a contaminação de cache do *Domain Name System* (DNS) utilizando domínios originais para saquear os dados do usuário em questão (Khonji; Iraqi; Jones, 2013).

2.4.2 Negação de Serviço Distribuído

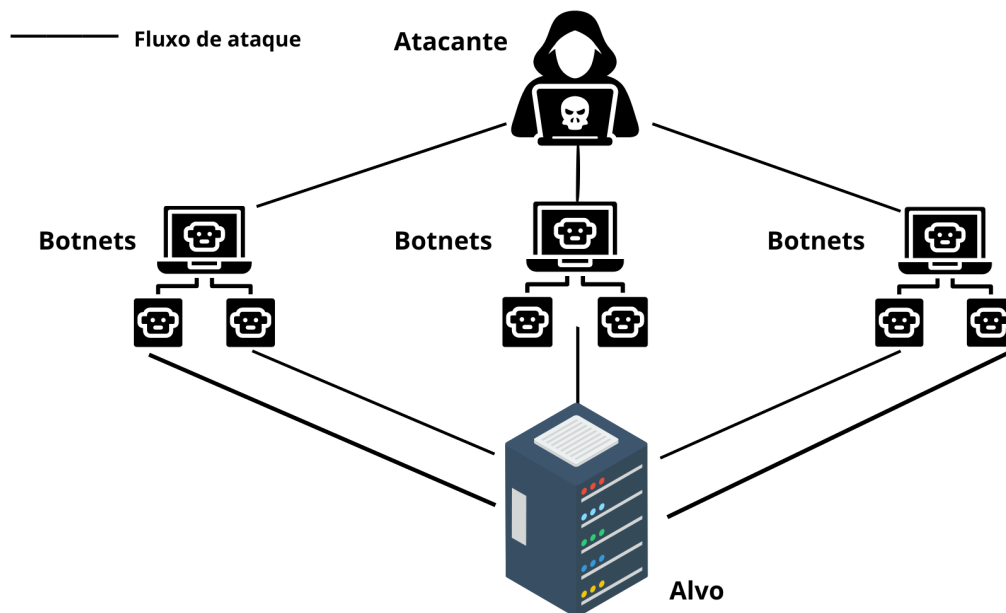
Em um ataque do tipo Negação de Serviço Distribuído - *Distributed Denial of Service* (DDoS) o invasor irá analisar os pontos fracos da rede onde é possível instalar *malwares* em diferentes máquinas e colocá-las sobre sua supervisão. Estes *hosts* infectados encaminham mais pacotes contaminados a outros *hosts* da mesma ou de outras redes (Hoque; Bhattacharyya; Kalita, 2015).

O DDoS tem capacidade de infectar rapidamente diversos *hosts* dependendo da quantidade de usuários comprometidos e da dimensão do pacote malicioso. Em um cenário deste tipo de ataque, existem três sujeitos principais:

- *Hacker*;
- *Botnet*;
- *Alvo*.

O sujeito que irá atacar o sistema é chamado de *hacker*. Ele comanda os dispositivos infectados chamados de *botnets*, que são responsáveis por gerar os volumes de tráfego de ataque a outras máquinas. Toda essa estrutura busca atacar o alvo, que nada mais é do que a organização, recurso ou infraestrutura de interesse do atacante (Patil; Deore, 2024). Um diagrama análogo a um ataque DDoS é representado pela Figura 2.4.

Figura 2.4 – Organização de um ataque DDoS.



Fonte: Adaptado de Patil e Deore (2024, on-line)

Quando o sistema ou rede se torna impossível de ser utilizado pelos administradores do sistema é quando ocorre a negação de serviço. Neste ponto, pode ser considerado um pagamento de resgate ao atacante ou mesmo a reconstrução de um ambiente através de *backups*.

2.4.3 *Ransomware*

O *Ransomware*, basicamente, trabalha com a criptografia dos dados do usuário afetado. Um bom exemplo seria o acesso do usuário a *websites* em busca de algum arquivo ou programa de interesse, logo, pode ocorrer o *download* de um *malware* que fará o trabalho silenciosamente. Outro exemplo de tentativa de ataque inicial seria um anexo, ou um *link* que leva a execução e instalação de um *malware* (Saxena; Soni, 2018).

No momento em que o vírus já está presente na máquina, o agente invasor analisa os arquivos pessoais da vítima e chama seu servidor para efetuar a criptografia dos dados. O principal método de criptografia utilizado é o *Rivest Shamir Adleman* (RSA), que além do primeiro ataque, pode se estender a dispositivos conectados e ao ambiente de rede (O’Kane; Sezer; Carlin, 2018).

2.5 Estado da Arte

Em um trabalho desenvolvido por Xian Guo, foi proposto uma arquitetura de *Zero Trust Intelligent Zero Trust Software Defined Networking* (IZTSDN) implantado em redes definidas por software. O aprendizado de máquina foi baseado na constituição de confiança zero, visando proteger os recursos e conexões da rede. O algoritmo efetua uma análise do comportamento dos usuários a todo momento e é capaz de reprimir o acesso de usuários invasores (Guo et al., 2023).

O autor modificou a distribuição do *Mininet* e simulou uma plataforma chamada de *MiniZTA*. Os resultados do trabalho provam que o rendimento do recurso é de 80,5% quando o cenário da rede é de ataques. Os números de detecção de tráfego fora do padrão vai a 99,56% de rendimento (Guo et al., 2023).

Pensando em uma solução para o mercado de digitalização dos sistemas industriais e uma grande demanda de tecnologias em nuvem, o autor Claudio Zanasi desenvolveu uma maneira de gestão segura destas estruturas. Sua implementação sugere uma arquitetura de segurança para sistemas *Industrial Internet of Things* (IIoT) (Zanasi; Russo; Colajanni, 2024).

O projeto mostra a efetivação e os resultados do SDN *peer-to-peer* em conjunto com o processo assíncrono de distribuições de políticas. As falhas particulares resultam em operações descentralizadas, o que gera uma administração flexível da topologia e das regras para um ambiente seguro (Zanasi; Russo; Colajanni, 2024).

Segundo Francis A. Ruambo, existem deficiências de segurança em ambientes SDN que projeta uma demanda de procedimentos de segurança adicionais. Para isso, o autor propôs um modelo de ataque e analisou o desempenho da solução sugerida. O trabalho ressalta implementações de confiança zero em SDN/NFV visando melhorar a segurança de dispositivos *endpoint* (Ruambo et al., 2023).

Uma pesquisa realizada por Naeem Syed mostra minuciosamente o paradigma de segurança *Zero Trust*, que tem apresentado crescimento de apoiadores no ramo de gerenciamento de infraestrutura. O trabalho apresenta opções de implementação de uma arquitetura de confiança zero e discute sobre técnicas de ponta de acesso em *Zero Trust Architectures* (ZTA) (Syed et al., 2022).

Além disso, o artigo propõe os desafios aos mecanismos de segurança como autenticação, controle, microsegmentação, entre outros. O trabalho é concluído com diferentes

caminhos que o *Zero Trust* pode seguir em linhas de pesquisa em ambientes com ambientes críticos (Syed et al., 2022).

Materiais e Métodos

A pesquisa abordada neste trabalho é do tipo experimental computacional, ou seja, será feita a partir de simulações computacionais com o intuito de investigar e explorar os objetivos declarados em capítulos anteriores. É definida como uma pesquisa mista, onde o objetivo combinar elementos de pesquisa qualitativa e quantitativa.

O trabalho é visto como misto pois busca avaliar o desempenho do sistema com dados numéricos mas também visa descobrir como o sistema tratado impactará os usuários. O público alvo que será investigado basicamente são todos os tipos de usuários e entidades (privadas ou públicas) que buscam maior segurança e implementação de tecnologias escaláveis e modeláveis.

Visando certa portabilidade, capacidade de exportação e importação rápida de um sistema em qualquer máquina com acesso à internet, o trabalho explora a inserção do projeto em contêineres. A partir desta premissa, todos os testes serão feitos dentro da plataforma containerizada.

Para verificar o desempenho da estrutura da SDN, é possível citar uma gama de testes, como:

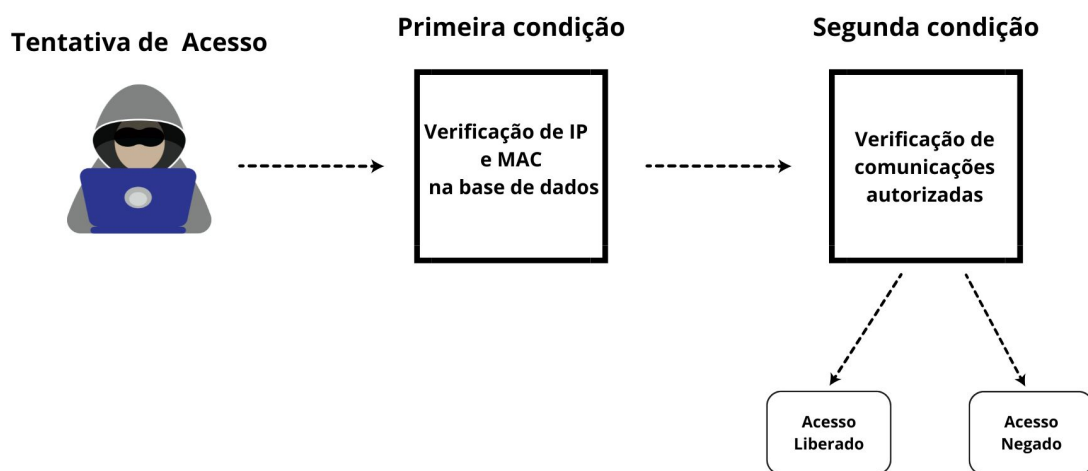
- Teste de conectividade através do comando *'ping'*;
- Cálculo da capacidade de transferência de dados com o protocolo TCP;
- Cálculo da capacidade de transferência de dados com o protocolo UDP;
- Verificação de tráfego de rede entre dispositivos;
- Verificação das regras de segurança implementadas.

Com a rede devidamente orquestrada e operante, a implementação dos princípios de segurança da arquitetura ZTNA será o próximo passo. Políticas de acesso serão criadas visando aumentar a barreira de segurança do sistema, para a prevenção de ataques. A arguição da seguridade do sistema terá um fluxo de testes que definirão o tipo de acesso que o usuário terá:

- Acesso total;
- Acesso limitado aos recursos;
- Acesso negado (*host* não autorizado).

As políticas de segurança propostas neste trabalho vão trabalhar conforme visto na Figura 3.1.

Figura 3.1 – Políticas de segurança da rede.



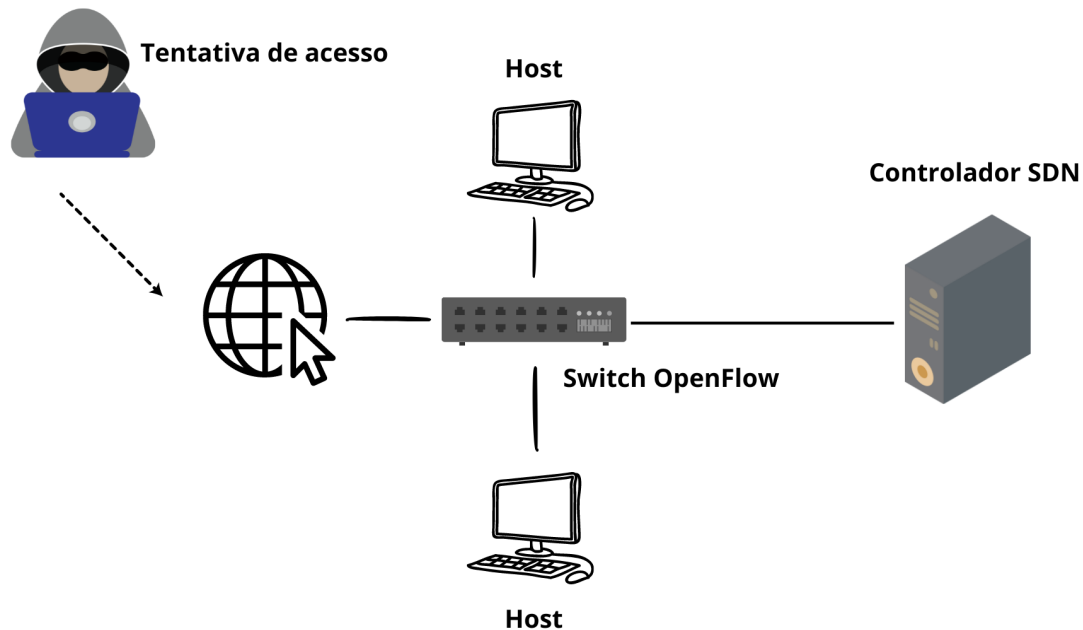
Fonte: O autor

Isso sugere que, diferentes usuários podem ter acesso a diferentes recursos sempre cumprindo às regras de encaminhamento que serão posteriormente propostas em códigos.

No primeiro contexto de acesso, o usuário cumpre todas as políticas de segurança impostas no sistema baseadas nos princípios de ZTNA e permissões de acesso. Esse acesso é definido como o acesso total. No segundo exemplo, com base nas políticas e suas características, o usuário terá um acesso limitado aos recursos do sistema.

Na última hipótese de acesso à rede, ocorre uma alusão a uma invasão, uma tentativa de ataque à rede. Esse acesso deve ser negado, mesmo que o *host* se encontre dentro da rede, pois não está em conformidade com as políticas de segurança estabelecidas. Todas as tentativas de acesso tem um fluxo de conexão como o ilustrado na Figura 3.2.

Figura 3.2 – Fluxo de uma tentativa de conexão a rede.



Fonte: O autor

3.1 Mininet

O Mininet é um emulador de redes definidas por software e também tem sua plataforma baseada em *Linux*. Ele utiliza a tecnologia de virtualização do kernel do Linux para criar hosts virtuais, *switches* e controladores SDN. Tudo isso pode ser feito em um único computador físico. Além disso, o *software* é capaz de emular essa rede, permitindo depuração e modelagem conforme necessário para atender aos objetivos do usuário.

A plataforma oficial do Mininet pode ser extraída através de um repositório no *GitHub*. Ao obter os arquivos, o *software* é clonado para uma pasta na raiz do sistema operacional do usuário.

3.2 Ryu

Para gerir uma SDN é necessário um controlador. O controlador neste trabalho deve ser compatível com o protocolo *OpenFlow*, assim como os dispositivos de encaminhamento. Neste trabalho será utilizado o *Ryu*, que é um gerente de redes de código aberto distribuído sobre a licença *Apache License 2.0*.

O *Ryu* pode ser obtido através do comando `'pip install ryu'` em uma linha de comando do *Linux* ou através da clonagem do seu repositório no *GitHub*.

Essa poderosa plataforma permite gerenciar fluxos de rede, monitorar o tráfego de dados que circula pela rede, criar regras de segurança e implementar filtragem de pacotes,

entre outras capacidades. Com essas funcionalidades, o Ryu possibilita a construção de um ambiente de rede personalizado, adaptável a diferentes necessidades e cenários (Bhardwaj; Panda, 2022).

O Ryu oferece uma interface de programação em Python, o que facilita a criação de aplicações customizadas e a integração com outras soluções. Sua modularidade permite que desenvolvedores escolham componentes específicos para suas implementações, tornando o processo de desenvolvimento mais ágil e eficiente (Chandroth; Roh; Ali, 2022).

3.3 *OpenFlow*

Desenvolvido pela Universidade de *Stanford* e pela *Open Networking Foundation*, o protocolo *OpenFlow* tem a função de fazer a comunicação entre o plano de controle e o plano de dados (Barbosa, 2018). Ele é capaz de gerenciar o comportamento dos dispositivos de encaminhamento de forma programável. Em seu ambiente, é possível definir regras de encaminhamento de pacotes, executar monitoramento do fluxo e a criação de políticas na rede local.

Com a programação do protocolo no dispositivo controlador, é definida uma interface de programação entre ele e os dispositivos de encaminhamento. Todo e qualquer dispositivo de encaminhamento compatível com o *OpenFlow* é capaz de seguir a lógica programada pelo controlador SDN.

3.4 *Docker*

Docker é um *software* compatível com *Linux* e *Windows* de código aberto. É uma ferramenta construída que visa facilitar a criação, implantação e execução de aplicativos, utilizando contêineres. Utilizado principalmente por desenvolvedores, *Docker* é capaz de funcionar de maneira isolada, comportando apenas a aplicação e suas dependências, assim, o usuário pode desenvolver sem se preocupar com o sistema operacional, por exemplo.

Outra vantagem sobre esse sistema é executar contêineres *Docker* sem sobrecarga de memória adicional, então vários contêineres executando vários serviços terão baixo consumo de recursos computacionais (Jangla; Jangla, 2018).

3.4.1 *Docker Compose*

O *Docker Compose* é uma ferramenta que permite definir e gerenciar aplicativos de múltiplos contêineres, ou seja é possível combinar dois ou mais contêineres. Isso pode ser feito com arquivo YAML que pode ser definido como uma combinação de vários contêineres *Dockerfile* executando preferências e configurações em um único arquivo (Jangla; Jangla,

2018). Este arquivo YAML do *Docker Compose* contém configurações de vários serviços. Para efetuar esta combinação basta executar três passos:

- Definir a configuração do contêiner em execução dentro de um arquivo *Docker*;
- Criar um arquivo YAML do *Docker Compose* que contenha configurações de todos os serviços que você deseja executar;
- Com comando 'docker-compose up', há a execução do arquivo YAML e a aplicação.

3.5 Instalação e configuração do *Docker Compose*

Em um ambiente *Linux* (Ubuntu 22.04), para dar início ao projeto, foi feita a instalação do *Docker* e do *Docker Compose* no sistema para que fosse possível ser importados os contêineres do *Mininet* e *Ryu*. Para isso, com o terminal do sistema operacional, foi redigido os códigos do Código 3.1.

Código 3.1 – Instalação do docker e Docker Compose

```
sudo apt install docker.io  
sudo apt install docker-compose
```

Para criar um diretório do projeto foi efetuado os comandos do Código 3.2.

Código 3.2 – Criação e abertura do diretório

```
mkdir sdn_project  
cd sdn_project
```

Com o diretório criado, é necessária a criação de um arquivo YAML que dará origem aos contêineres combinados. Dentro deste arquivo são definidas todas as suas características, configurações e preferências. O Código 3.3 mostra como foi definido.

Código 3.3 – Definições do Docker Compose

```
version: '3'  
services:  
  ryu:  
    image: docker.io/lataarc/ryu  
    privileged: true  
    networks:  
      - mynetwork  
  mininet:
```

```

        image: docker.io/iwaseyusuke/mininet:ubuntu-22.04   #
            latest
        tty: true
        privileged: true
        environment:
            - DISPLAY
        volumes:
            - /tmp/.X11-unix:/tmp/.X11-unix
            - /lib/modules:/lib/modules
        networks:
            - mynetwork

networks:
    mynetwork:
        external: true
        name: igor_mynetwork

```

3.6 Configuração e importação dos arquivos *Python* para o *Ryu*

Como dito anteriormente, serão utilizados dois arquivos *Python* para o controlador *Ryu*. O primeiro arquivo será responsável por gerenciar uma topologia simples para testes de comunicação e o segundo arquivo terá a adição de regras de segurança.

O arquivo que trabalha com o teste de gerenciamento e comunicação foi definido como escrito no Código 3.4.

Código 3.4 – Definições do arquivo de comunicação simples

```

from ryu.base import app_manager
from ryu.controller import ofp_event
from ryu.controller.handler import MAIN_DISPATCHER
from ryu.controller.handler import set_ev_cls
from ryu.ofproto import ofproto_v1_0
from ryu.lib.packet import packet
from ryu.lib.packet import ethernet, ipv4

class L2Switch(app_manager.RyuApp):
    OFP_VERSIONS = [ofproto_v1_0.OFP_VERSION]

    def __init__(self, *args, **kwargs):
        super(L2Switch, self).__init__(*args, **kwargs)

    set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
    def packet_in_handler(self, ev):

```

```

msg = ev.msg
dp = msg.datapath
ofp = dp.ofproto
ofp_parser = dp.ofproto_parser

# Logando o evento de PacketIn
self.logger.info("PacketIn recebido: %s", msg)

pkt = packet.Packet(msg.data)
eth = pkt.get_protocol(ethernet.ethernet)

# Verifica se o pacote e do tipo IPv4
if eth.ethertype == 0x0800: # IPv4
    ip_pkt = pkt.get_protocol(ipv4.ipv4)
    src_ip = ip_pkt.src
    dst_ip = ip_pkt.dst
    in_port = msg.in_port

# Logando informacoes de origem, destino e porta de entrada
self.logger.info("Pacote recebido de %s na porta %d, destino
    %s", src_ip, in_port, dst_ip)

actions = [ofp_parser.OFPActionOutput(ofp.OFPP_FLOOD)]

data = None
if msg.buffer_id == ofp.OFP_NO_BUFFER:
    data = msg.data

out = ofp_parser.OFPPacketOut(
    datapath=dp, buffer_id=msg.buffer_id, in_port=msg.in_port
    ,
    actions=actions, data=data)
dp.send_msg(out)

```

E, por fim, o arquivo com as regras de segurança foi escrito da seguinte forma listada no Código 3.5.

Código 3.5 – Definições do arquivo com regras de segurança

```

import logging
from ryu.base import app_manager
from ryu.controller import ofp_event
from ryu.controller.handler import MAIN_DISPATCHER, CONFIG_DISPATCHER
from ryu.controller.handler import set_ev_cls
from ryu.ofproto import ofproto_v1_3
from ryu.lib.packet import packet
from ryu.lib.packet import ethernet, ipv4

```



```

        datapath.send_msg(mod)
        self.logger.info("Adicionando fluxo: prioridade=%d, match=%s,
                          a es=%s", priority, match, actions)

    set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
    def packet_in_handler(self, ev):
        msg = ev.msg
        datapath = msg.datapath
        ofproto = datapath.ofproto
        parser = datapath.ofproto_parser
        in_port = msg.match['in_port']

        pkt = packet.Packet(msg.data)
        eth = pkt.get_protocols(ethernet.ethernet)[0]

        # Ignorar pacotes LLDP
        if eth.ethertype == ether_types.ETH_TYPE_LLDP:
            return

        # Extracao de informacoes do pacote
        pkt_ipv4 = pkt.get_protocol(ipv4.ipv4)
        src_ip = pkt_ipv4.src if pkt_ipv4 else None
        dst_ip = pkt_ipv4.dst if pkt_ipv4 else None

        # Logar pacotes entre h1 e h2
        if (src_ip == '10.0.0.1' and dst_ip == '10.0.0.2') or (src_ip
            == '10.0.0.2' and dst_ip == '10.0.0.1'):
            self.logger.info("Pacote permitido: %s -> %s", src_ip,
                             dst_ip)
        else:
            self.logger.info("Pacote n o permitido: %s -> %s",
                             src_ip, dst_ip)
            return # Dropar o pacote

```

3.7 Configuração do *Mininet*

O *software* do *Mininet* conta com diversos exemplos de topologias que podem ser executadas a qualquer momento na aplicação. Em questão organizacional e experimental, para validar a programação de novas redes conforme a necessidade do usuário, foi implementado dois arquivos *Python* para definir os *hosts*, o *switch* e a existência e conexão do controlador já definido.

Foram necessárias duas topologias diferentes, pois, para o primeiro experimento, o protocolo *Openflow* na versão 1.0 bastava. Já para o teste onde foram implementadas

regras de segurança, é necessário a utilização da versão 1.3 do protocolo além do *switch* no *Mininet* que deve ser definido em sua versão compatível com tais.

O Código 3.6 representa o código utilizado para a comunicação simples entre hosts.

Código 3.6 – Topologia utilizada em código

```
from mininet.net import Mininet
from mininet.node import RemoteController
from mininet.topo import Topo
from mininet.log import setLogLevel
from mininet.cli import CLI

class CustomTopo(Topo):
    def build(self):
        # Adiciona tres hosts
        h1 = self.addHost('h1', ip='10.0.0.1/24')
        h2 = self.addHost('h2', ip='10.0.0.2/24')
        h3 = self.addHost('h3', ip='10.0.0.3/24')

        # Adiciona um switch
        s1 = self.addSwitch('s1')

        # Conecta os hosts ao switch
        self.addLink(h1, s1)
        self.addLink(h2, s1)
        self.addLink(h3, s1)

def run():
    # Configuracao do controlador remoto
    ryu_controller = RemoteController('ryu', ip='10.89.0.2', port
                                     =6653)

    # Criacao da rede com a topologia personalizada
    net = Mininet(topo=CustomTopo(), controller=ryu_controller)

    # Inicia a rede
    net.start()

    # Abre o CLI do Mininet para testes e comandos interativos
    CLI(net)

    # Para a rede ao sair do CLI
    net.stop()

if __name__ == '__main__':
    setLogLevel('info')
    run()
```

Já no Código 3.7 trata-se de uma adaptação para que funcione com o arquivo que engloba as regras de segurança.

Código 3.7 – Código adaptado para o switch OpenFlow 1.3

```
from mininet.net import Mininet
from mininet.node import Controller, RemoteController, OVSSwitch
from mininet.topo import Topo
from mininet.log import setLogLevel
from mininet.cli import CLI

class CustomTopo(Topo):
    def build(self):
        # Adiciona tres hosts com IPs e MACs especificos
        h1 = self.addHost('h1', ip='10.0.0.1', mac='00:00:00:00:00:01'
            ')
        h2 = self.addHost('h2', ip='10.0.0.2', mac='00:00:00:00:00:02'
            ')
        h3 = self.addHost('h3', ip='10.0.0.3', mac='00:00:00:00:00:03'
            ')

        # Adiciona um switch e configura para OpenFlow 1.3
        s1 = self.addSwitch('s1', protocols='OpenFlow13')

        # Conecta os hosts ao switch
        self.addLink(h1, s1)
        self.addLink(h2, s1)
        self.addLink(h3, s1)

def run():
    # Configuracao do controlador remoto
    ryu_controller = RemoteController('ryu', ip='10.89.0.2', port
        =6653)

    # Criacao da rede com a topologia personalizada
    net = Mininet(topo=CustomTopo(), controller=ryu_controller)

    # Inicia a rede
    net.start()

    # Abre o CLI do Mininet para testes e comandos interativos
    CLI(net)

    # Para a rede ao sair do CLI
    net.stop()
```

```

|| if __name__ == '__main__':
||     setLogLevel('info')
||     run()

```

3.8 Execução dos contêineres

Para iniciar o *Docker Compose* criado em segundo plano é necessário o seguinte comando do Código 3.8.

Código 3.8 – Execução da aplicação

```

|| docker-compose up -d

```

É possível verificar se a requisição foi concluída corretamente digitando o Código 3.9.

Código 3.9 – Verificação dos contêineres

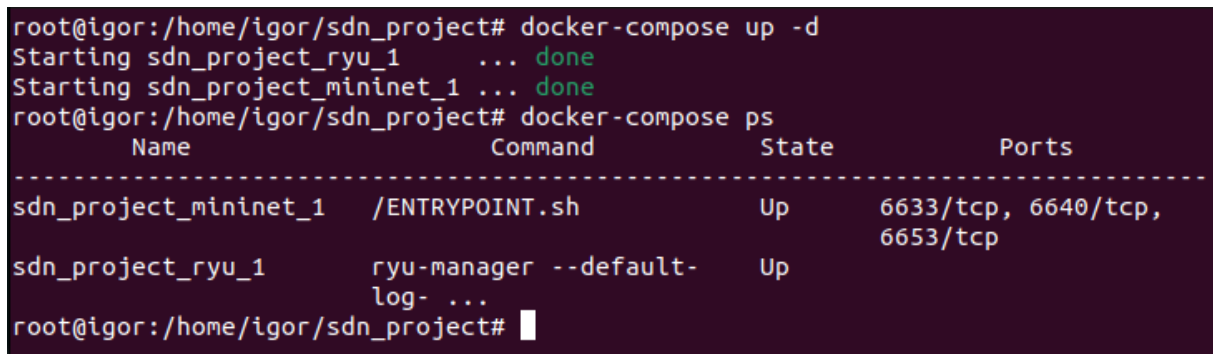
```

|| docker-compose ps

```

Assim, o terminal mostra o nome, o comando, o estado e as portas dos contêineres contidos no sistema como representado pela Figura 3.3.

Figura 3.3 – Listagem e informações dos contêineres.



```

root@igor:/home/igor/sdn_project# docker-compose up -d
Starting sdn_project_ryu_1      ... done
Starting sdn_project_mininet_1 ... done
root@igor:/home/igor/sdn_project# docker-compose ps

```

Name	Command	State	Ports
sdn_project_mininet_1	/ENTRYPOINT.sh	Up	6633/tcp, 6640/tcp, 6653/tcp
sdn_project_ryu_1	ryu-manager --default-log- ...	Up	

```

root@igor:/home/igor/sdn_project#

```

Fonte: O autor.

3.9 Movendo os arquivos *Python* criados para os respectivos contêineres

Em terminais diferentes do *Linux* precisam ser abertas as aplicações do controlador e do emulador de SDN. Em um dos terminais, para abrir inicialmente o *Ryu* é escrito o Código 3.10.

Código 3.10 – Execução do terminal de interação do Ryu

```

|| docker exec -it sdn_project_ryu_1 /bin/sh

```

Logo, para abrir o *Mininet*, similarmente se insere o Código 3.11.

Código 3.11 – Execução do terminal de interação do Mininet

```
|| docker exec -it sdn_project_mininet_1 /bin/bash
```

Os contêineres precisam de um processo diferente para mover arquivos para dentro dos mesmos. Partindo de um arquivo contido no *host* de simulação para um diretório do contêiner é necessário especificar o caminho do arquivo no *host* e para onde ele irá dentro da aplicação.

Para que os arquivos criados fossem executados nos contêineres eles foram adicionados com os comandos do Código 3.12 em outro terminal.

Código 3.12 – Transferência dos arquivos para os contêineres

```
#Copiando o arquivo Python do Ryu de simples conexao entre hosts
docker cp /home/igor/sdn_project/l2.py sdn_project_ryu_1:/usr/local/
lib/python3.8/site-packages/ryu/app

#Copiando o arquivo Python do Ryu com regras de seguranca
docker cp /home/igor/sdn_project/zero_trust sdn_p oject_ryu_1:/usr/
local/lib/python3.8/site-packages/ryu/app

#Copiando a topologia do protocolo OpenFlow v1.0 que sera utilizada
no Mininet
docker cp /home/igor/sdn_project/my_topo_ryu.py sdn_project_mininet_1
:/home

#Copiando a topologia do protocolo OpenFlow v1.3 que sera utilizada
no Mininet
docker cp /home/igor/sdn_project/my_topo_ryu_opfw.py
sdn_project_mininet_1:/home
```

Resultados

4.1 Abertura do arquivo "l2.py" e topologia do *Mininet*

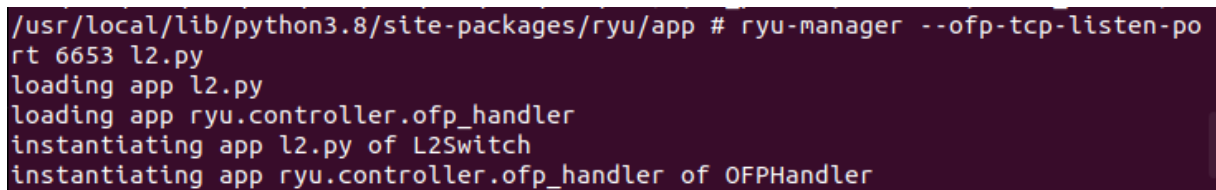
Para abrir o arquivo *Python* de gerenciamento e comunicação do *Ryu* foi executado em seu terminal o comando contido no Código 4.1.

Código 4.1 – Execução do arquivo de comunicação

```
|| ryu-manager --ofp-tcp-listen-port 6653 l2.py
```

Com sucesso, o terminal do *Ryu* exibe as informações contidas na Figura 4.1

Figura 4.1 – Resultado da execução do arquivo "l2.py".



```
/usr/local/lib/python3.8/site-packages/ryu/app # ryu-manager --ofp-tcp-listen-po  
rt 6653 l2.py  
loading app l2.py  
loading app ryu.controller.ofp_handler  
instantiating app l2.py of L2Switch  
instantiating app ryu.controller.ofp_handler of OFPHandler
```

Fonte: O autor.

De maneira similar, no *bash* do *Mininet*, para abrir a topologia descrita anteriormente foi redigido o Código 4.2.

Código 4.2 – Execução da topologia

```
|| python3 my_topo_ryu.py
```

Em resposta, o terminal do *Mininet* cria os *hosts*, o *switch*, os *links* e efetua a conexão com o controlador conforme visto pela Figura 4.2.

Figura 4.2 – Topologia criada pelo *script* do *Mininet*.

```
root@b515f68f5280:/home# python3 my_topo_ryu.py
*** Error setting resource limits. Mininet's performance may be affected.
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1)
*** Configuring hosts
h1 h2 h3
*** Starting controller
ryu
*** Starting 1 switches
s1 ...
*** Starting CLI:
```

Fonte: O autor.

O *software* mostra que foram criados 3 *hosts*, um *switch OpenFlow* e foi adicionado os *links*, que são as conexões entre os *hosts* e o *switch*. Consta também no terminal, a inicialização do controlador configurado anteriormente e do *switch*.

4.2 Testando a conectividade da topologia

Para validar que o *Mininet* e o *Ryu* estavam se comunicando devidamente foi testado o comando que envia pacotes do tipo *Internet Control Message Protocol* (ICMP) no *bash* do *Mininet* listado no Código 4.3.

Código 4.3 – Envio de pacotes de teste

```
|| pingall
```

E o resultado deve ser conforme mostrado pelas Figuras 4.3 e 4.4.

Figura 4.3 – Resultado do ping do primeiro arquivo no terminal do *Ryu*.

```
'\x0e\x1c\x00\x00\x00\x003\xdd\x0c\x00\x00\x00\x00\x00\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f !"#%&\'()*+,-./01234567',in_port=3,reason=0,total_len=98)
Pacote recebido de 10.0.0.3 na porta 3, destino 10.0.0.2
PacketIn recebido: version=0x1,msg_type=0xa,msg_len=0x74,xid=0x0,OFPPacketIn(buffer_id=4294967295,data=b'\x1a\x8c\x03\x9eVG\x8a\xe6q\xea\x09\x18\x08\x00E\x00\x00T\xdd\xb7\x00\x00@\x01\x88\xed\n\x00\x00\x02\n\x00\x00\x03\x00\x00%q\x98h\x00\x01'\x0e\x1c\x00\x00\x00\x003\xdd\x0c\x00\x00\x00\x00\x00\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f !"#%&\'()*+,-./01234567',in_port=2,reason=0,total_len=98)
Pacote recebido de 10.0.0.2 na porta 2, destino 10.0.0.3
PacketIn recebido: version=0x1,msg_type=0xa,msg_len=0x58,xid=0x0,OFPPacketIn(buffer_id=4294967295,data=b'33\x00\x00\x00\x02&M}\x0f\xe2\t\x86\xdd'\x00\x00\x00\x00\x10:\xff\xfe\x80\x00\x00\x00\x00\x00\x00$M}\xff\xfe\x0f\xe2\t\xff\x02\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x02\x85\x00ta\x00\x00\x00\x00\x01\x01&M}\x0f\xe2\t',in_port=1,reason=0,total_len=70)
PacketIn recebido: version=0x1,msg_type=0xa,msg_len=0x58,xid=0x0,OFPPacketIn(buffer_id=4294967295,data=b'33\x00\x00\x00\x02\x8a\xe6q\xea\x09\x18\x86\xdd'\x00\x00\x00\x00\x10:\xff\xfe\x80\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x02\x85\x00\x00\x00\x00\x01\x01\x8a\xe6q\xea\x09\x18',in_port=2,reason=0,total_len=70)
PacketIn recebido: version=0x1,msg_type=0xa,msg_len=0x58,xid=0x0,OFPPacketIn(buffer_id=4294967295,data=b'33\x00\x00\x00\x02\x1a\x8c\x03\x9eVG\x86\xdd'\x00\x00\x00\x00\x10:\xff\xfe\x80\x00\x00\x00\x00\x00\x00\x00\x18\x8c\x03\xff\xfe\x9eVG\xff\x00
```

Fonte: O autor.

De acordo com o que é visualizado no terminal, podemos notar que os *logs* mostram várias linhas detalhadas sobre pacotes do tipo *PacketIn*, que são mensagens enviadas dos *switches* para o controlador. Essas mensagens indicam que um pacote não pôde ser tratado diretamente pelo *switch*, e ele está pedindo instruções ao controlador.

A informação "version=0x1" indica que a versão do *OpenFlow* utilizada é a versão 1.0, já a "msg_type=0xa" informa que a mensagem é do tipo *PacketIn*. O campo "msg_len=0x58" mostra o tamanho da mensagem (em *bytes*) e o "xid=0x..." mostra o identificador da transação para correlacionar a resposta.

Em sequência o "buffer_id=..." entrega o identificador do pacote que fica no *buffer* do *switch* até o controlador decidir o que fazer com o mesmo. O campo "data=..." são os dados brutos do pacote e o "total_len=70" contabiliza o tamanho total do pacote em *bytes*. Há também a informação da porta em "in_port= 1" indicando que o pacote é recebido na porta 1 do *switch*.

Figura 4.4 – Resultado do ping do primeiro arquivo no terminal do *Mininet*.

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3
h2 -> h1 h3
h3 -> h1 h2
*** Results: 0% dropped (6/6 received)
```

Fonte: O autor.

Já no *bash* do *Mininet*, consta que 6/6 pacotes foram devidamente recebidos de um para todos os outros *hosts*.

4.3 Capacidade de transferência de dados dos protocolos TCP e UDP

Para o cálculo da velocidade de transferência de dados para o protocolo de comunicação TCP, o *Mininet* dispõe da função "iperf" que faz essa estimativa. Seu resultado pode ser visto na Figura 4.5.

Figura 4.5 – Capacidade de transferência de dados do protocolo TCP para o arquivo "l2.py".

```
mininet> iperf
*** Iperf: testing TCP bandwidth between h1 and h3
*** Results: ['15.7 Mbits/sec', '15.5 Mbits/sec']
```

Fonte: O autor.

O resultado mostra que a velocidade de comunicação entre os pontos de h1 até h3 no protocolo TCP é de 15,7 Mbits/s e que de h3 a h1 tem 15,5Mbits/s. Similarmente, para o mesmo teste com o protocolo de comunicação UDP, a função "iperfudp" tem o resultado representado pela Figura 4.6.

Figura 4.6 – Capacidade de transferência de dados do protocolo UDP para o arquivo "l2.py".

```
mininet> iperfudp
*** Iperf: testing UDP bandwidth between h1 and h3
*** Results: ['10M', '10.5 Mbits/sec', '10.5 Mbits/sec']
```

Fonte: O autor.

Nos testes utilizando o protocolo UDP mostra inicialmente a taxa alvo determinada em 10 Mbits/s e depois as taxas de 10,5 e 10,5 Mbits/s, bilateralmente entre h1 e h3.

É importante ressaltar que a tecnologia utilizada neste primeiro experimento inclui a versão 1.0 do protocolo *OpenFlow*, lançada em meados do ano de 2011. Isso pode justificar a velocidade limitada quando comparada a uma *Local Area Network* (LAN) convencional física, além do contexto de um ambiente virtualizado.

4.4 Teste das regras de segurança

Para abrir o arquivo *Python* que contém as regras de segurança propostas, no terminal do controlador, foi redigido o Código 4.4.

Código 4.4 – Execução do arquivo com regras de segurança

```
|| ryu-manager --ofp-tcp-listen-port 6653 zero_trust.py
```

O terminal do *Ryu* representado pela Figura 4.7 mostra que o aplicativo foi carregado aparentemente sem erros.

Figura 4.7 – *Logs* da abertura do arquivo.

```
/usr/local/lib/python3.8/site-packages/ryu/app # ryu-manager --ofp-tcp-listen-po  
rt 6653 zero_trust.py  
loading app zero_trust.py  
loading app ryu.controller.ofp_handler  
instantiating app zero_trust.py of ZeroTrustSDN  
instantiating app ryu.controller.ofp_handler of OFPHandler
```

Fonte: O autor.

Partindo para o carregamento da topologia compatível com o código do controlador carregado, no terminal do emulador de redes, foi executado o Código 4.5.

Código 4.5 – Execução da topologia com switch OpenFlow

```
|| python3 my_topo_ryu_opfw.py
```

Com isso, a saída do *Mininet* exibe os *logs* mostrados na Figura 4.8.

Figura 4.8 – *Logs* do carregamento da topologia

```
*** Creating network  
*** Adding controller  
*** Adding hosts:  
h1 h2 h3  
*** Adding switches:  
s1  
*** Adding links:  
(h1, s1) (h2, s1) (h3, s1)  
*** Configuring hosts  
h1 h2 h3  
*** Starting controller  
ryu  
*** Starting 1 switches  
s1 ...  
*** Starting CLI:
```

Fonte: O autor.

Assim que a topologia é carregada, o *Ryu* mostra as regras de fluxo representadas pela Figura 4.9.

Figura 4.9 – Logs do Ryu.

```

Adicionando fluxo: prioridade=1, match=OFPMatch(oxm_fields={'in_port': 1, 'eth_t
ype': 2048, 'ipv4_src': '10.0.0.1', 'ipv4_dst': '10.0.0.2'}), ações=[OFPActionOu
tput(len=16,max_len=65509,port=2,type=0)]
Adicionando fluxo: prioridade=1, match=OFPMatch(oxm_fields={'in_port': 2, 'eth_t
ype': 2048, 'ipv4_src': '10.0.0.2', 'ipv4_dst': '10.0.0.1'}), ações=[OFPActionOu
tput(len=16,max_len=65509,port=1,type=0)]
Adicionando fluxo: prioridade=1, match=OFPMatch(oxm_fields={'in_port': 3}), açõe
s=[]
Adicionando fluxo: prioridade=0, match=OFPMatch(oxm_fields={}), ações=[OFPAction
Output(len=16,max_len=65509,port=4294967291,type=0)]

```

Fonte: O autor.

Portanto, com os devidos códigos carregados, pode-se testar as regras de encaminhamento definidas anteriormente. Inicialmente, foi feito um teste do *host* 1 (h1) para o *host* 2 (h2) e vice-versa, como visto nas Figuras 4.10 e 4.11.

Figura 4.10 – Teste de h1 para h2.

```

mininet> h1 ping h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.343 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.055 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.035 ms
^C

```

Fonte: O autor.

Figura 4.11 – Teste de h2 para h1.

```

mininet> h2 ping h1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=0.195 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=0.034 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=0.045 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=0.033 ms
64 bytes from 10.0.0.1: icmp_seq=5 ttl=64 time=0.048 ms
^C
--- 10.0.0.1 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4112ms
rtt min/avg/max/mdev = 0.033/0.071/0.195/0.062 ms

```

Fonte: O autor.

Concluindo que há conexão entre eles, como se era esperado.

Em sequência foi feito o teste de conexão entre o *host* 1 e o *host* 3 e vice-versa, conforme visto pelas Figuras 4.12 e 4.13.

Figura 4.12 – Teste de h1 para h3.

```
mininet> h1 ping h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
From 10.0.0.1 icmp_seq=1 Destination Host Unreachable
From 10.0.0.1 icmp_seq=2 Destination Host Unreachable
From 10.0.0.1 icmp_seq=3 Destination Host Unreachable
From 10.0.0.1 icmp_seq=4 Destination Host Unreachable
From 10.0.0.1 icmp_seq=5 Destination Host Unreachable
From 10.0.0.1 icmp_seq=6 Destination Host Unreachable
From 10.0.0.1 icmp_seq=7 Destination Host Unreachable
From 10.0.0.1 icmp_seq=8 Destination Host Unreachable
From 10.0.0.1 icmp_seq=9 Destination Host Unreachable
From 10.0.0.1 icmp_seq=10 Destination Host Unreachable
From 10.0.0.1 icmp_seq=11 Destination Host Unreachable
From 10.0.0.1 icmp_seq=12 Destination Host Unreachable
^C
--- 10.0.0.3 ping statistics ---
13 packets transmitted, 0 received, +12 errors, 100% packet loss, time 12321ms
pipe 4
```

Fonte: O autor.

Nota-se que não há conexão entre h1 e h3, visualizado como 0 pacotes recebidos.

Figura 4.13 – Teste de h3 para h1.

```
mininet> h3 ping h1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
From 10.0.0.3 icmp_seq=9 Destination Host Unreachable
From 10.0.0.3 icmp_seq=13 Destination Host Unreachable
From 10.0.0.3 icmp_seq=14 Destination Host Unreachable
From 10.0.0.3 icmp_seq=15 Destination Host Unreachable
From 10.0.0.3 icmp_seq=16 Destination Host Unreachable
From 10.0.0.3 icmp_seq=17 Destination Host Unreachable
From 10.0.0.3 icmp_seq=18 Destination Host Unreachable
From 10.0.0.3 icmp_seq=19 Destination Host Unreachable
From 10.0.0.3 icmp_seq=20 Destination Host Unreachable
From 10.0.0.3 icmp_seq=22 Destination Host Unreachable
From 10.0.0.3 icmp_seq=23 Destination Host Unreachable
^C
--- 10.0.0.1 ping statistics ---
24 packets transmitted, 0 received, +11 errors, 100% packet loss, time 23573ms
pipe 4
```

Fonte: O autor.

Os resultados mostram que a tentativa de conexão entre os *hosts* exibe a mensagem "*Destination Host Unreachable*" que informa que o endereço está inalcançável. A reação do sistema está de acordo com a política configurada, impedindo a conexão do *host* não permitido na rede.

Por fim, foi feito o teste de conexão entre o *host* 2 e o *host* 3, como visualizado nas Figuras 4.14 e 4.15.

Figura 4.14 – Teste de h2 para h3.

```
mininet> h2 ping h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
From 10.0.0.2 icmp_seq=1 Destination Host Unreachable
From 10.0.0.2 icmp_seq=2 Destination Host Unreachable
From 10.0.0.2 icmp_seq=3 Destination Host Unreachable
From 10.0.0.2 icmp_seq=4 Destination Host Unreachable
From 10.0.0.2 icmp_seq=5 Destination Host Unreachable
From 10.0.0.2 icmp_seq=6 Destination Host Unreachable
^C
--- 10.0.0.3 ping statistics ---
8 packets transmitted, 0 received, +6 errors, 100% packet loss, time 7205ms
pipe 4
```

Fonte: O autor.

Como esperado, não há comunicação entre os *hosts* h2 e h3. De 8 pacotes transmitidos, 0 foram recebidos, ou seja, 100% dos pacotes foram descartados.

Figura 4.15 – Teste de h3 para h2.

```
mininet> h3 ping h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
From 10.0.0.3 icmp_seq=9 Destination Host Unreachable
From 10.0.0.3 icmp_seq=10 Destination Host Unreachable
From 10.0.0.3 icmp_seq=11 Destination Host Unreachable
From 10.0.0.3 icmp_seq=13 Destination Host Unreachable
From 10.0.0.3 icmp_seq=14 Destination Host Unreachable
From 10.0.0.3 icmp_seq=15 Destination Host Unreachable
From 10.0.0.3 icmp_seq=16 Destination Host Unreachable
From 10.0.0.3 icmp_seq=17 Destination Host Unreachable
^C
--- 10.0.0.2 ping statistics ---
18 packets transmitted, 0 received, +8 errors, 100% packet loss, time 17420ms
pipe 4
mininet> 
```

Fonte: O autor.

Similarmente, com a comunicação de h3 para h2, não há transferência de pacotes, uma vez que estes são tratados pelo controlador que aplica as regras de segurança definidas no arquivo *Python*.

4.5 Resultados de banda para protocolo TCP e UDP com as políticas de segurança

Para efetuar os testes de capacidade de transferência de dados com o arquivo de segurança foi necessário algumas alterações. Como neste cenário a política de segurança não permite a comunicação com o terceiro *host* (h3), o teste precisa ser feito entre os *hosts* h1 e h2. Para isso, é necessário direcionar o teste de banda através da função entre os *hosts* comunicáveis.

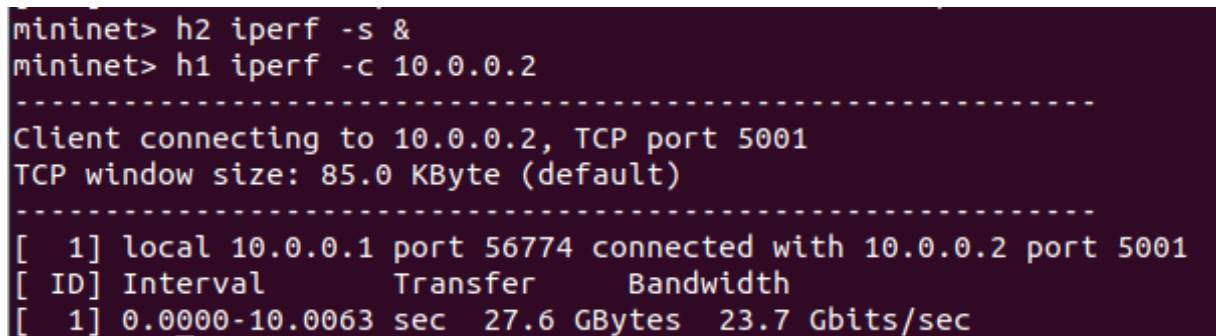
O Código 4.6 é feito para que o teste seja feito de forma correta.

Código 4.6 – Cálculo de banda entre h1 e h2 no TCP

```
h2 iperf -s &  
h1 iperf -c 10.0.0.2
```

Assim, o resultado do cálculo pode ser visto na Figura 4.16.

Figura 4.16 – Capacidade de transferência de dados com o protocolo TCP com as regras de segurança.



```
mininet> h2 iperf -s &  
mininet> h1 iperf -c 10.0.0.2  
-----  
Client connecting to 10.0.0.2, TCP port 5001  
TCP window size: 85.0 KByte (default)  
-----  
[  1] local 10.0.0.1 port 56774 connected with 10.0.0.2 port 5001  
[ ID] Interval           Transfer     Bandwidth  
[  1] 0.0000-10.0063 sec   27.6 GBytes 23.7 Gbits/sec
```

Fonte: O autor.

O terminal indica que a quantidade de dados transferida entre os *hosts* h1 e h2 no protocolo de comunicação TCP é de 27,6 GBytes e que sua banda atinge 23,7 Gbits/s.

Para o teste com a função que aplica o UDP, é redigido os comandos do Código 4.7.

Código 4.7 – Cálculo de banda entre h1 e h2 no UDP

```
h2 iperf -s &  
h1 iperf -c 10.0.0.2 -u -b 1G
```

Neste cenário, é descrita uma *flag* que indica o limite que a banda pode atingir. Isso pode ser visto como uma excelente configuração onde redes precisam ter um fluxo reduzido. O resultado deste teste pode ser visualizado pela Figura 4.17.

Figura 4.17 – Capacidade de transferência de dados com o protocolo UDP com as regras de segurança.

```
mininet> h2 iperf -s &
-----
Server listening on TCP port 5001
TCP window size: 128 KByte (default)
-----
mininet> h1 iperf -c 10.0.0.2 -u -b 1G
-----
Client connecting to 10.0.0.2, UDP port 5001
Sending 1470 byte datagrams, IPG target: 10.95 us (kalman adjust)
UDP buffer size: 208 KByte (default)
-----
[  1] local 10.0.0.1 port 54837 connected with 10.0.0.2 port 5001
[ ID] Interval           Transfer     Bandwidth
[  1] 0.0000-10.0001 sec   1.25 GBytes 1.07 Gbits/sec
[  1] Sent 913055 datagrams
```

Fonte: O autor.

Já com o protocolo UDP, o sistema indica 1,25 Gbytes transferidos entre os *hosts* e uma taxa de 1,07 Gbits/s. Como justificado, a taxa se comportou como o esperado, se limitando próximo ao limite anteriormente especificado.

Considerações Finais

A pesquisa representa a síntese dos esforços empreendidos para projetar, implementar e avaliar uma rede definida por software com base nos princípios de segurança da arquitetura *Zero Trust Network Access*. Ao longo deste estudo, foi possível alcançar uma série de objetivos e resultados significativos.

A rede definida por software em um ambiente containerizado mostrou bom funcionamento e foi capaz de realizar todos os experimentos de maneira rápida e isolada. Por estar neste tipo de virtualização, o ambiente tem facilidade de exportação e importação da nuvem pela comunidade *Docker*, que pode ser ideal para diferentes tipos de uso.

As regras de segurança impostas obtiveram bons resultados por conseguir isolar a comunicação de um *host* por mais que ele tenha sido definido na topologia previamente determinada no *Mininet*. Isso é visto como um avanço, pois, pensando em contextos mais complexos onde existe a necessidade da análise de tráfego e comunicação em redes definidas por software, a segurança do sistema pode ser altamente elevada através da criação e implementação de políticas de segurança.

A implementação da rede SDN e das medidas de segurança baseadas em ZTNA tenta apresentar um marco significativo, que pode proporcionar uma estrutura flexível e ágil para a configuração dinâmica e programática dos recursos de rede. Os testes de funcionalidade e segurança realizados visam confirmar a eficácia dos princípios de autenticação contínua, autorização granular e segmentação de rede na proteção contra ameaças internas e externas.

Além disso, a documentação detalhada produzida ao longo do trabalho, juntamente com as futuras recomendações e melhorias, servirá como um recurso valioso para profissionais e pesquisadores interessados na implementação de redes seguras e escaláveis.

Em suma, este trabalho faz uma tentativa de dar um passo significativo em direção à construção de redes de próxima geração que priorizam a segurança, a flexibilidade e o desempenho. Ao integrar os princípios de ZTNA em uma arquitetura SDN, o projeto visa demonstrar que é possível criar ambientes de rede confiáveis e resilientes, capazes de atender às demandas de um mundo digital em constante evolução.

Referências

- ALCARAZ, Cristina; LOPEZ, Javier; WOLTHUSEN, Stephen. Policy enforcement system for secure interoperable control in distributed smart grid systems. **Journal of Network and Computer Applications**, v. 59, p. 301–314, 2016. ISSN 1084-8045. Disponível em: <https://www.sciencedirect.com/science/article/pii/S1084804515001629>.
- ALEROUD, Ahmed; ZHOU, Lina. Phishing environments, techniques, and countermeasures: A survey. **Computers & Security**, Elsevier, v. 68, p. 160–196, 2017.
- ATLÂNTICO. **Redes definidas por software: tudo que você precisa saber sobre SDN**. 2017. Disponível em: <https://www.atlantico.com.br/blog/plataformas-tecnologicas/redes-definidas-por-software-tudo-que-voce-precisa-saber-sobre-sdn>. Acessado em 24 jan. 2024.
- BARBOSA, Renan Rodrigo. **Migração de redes tradicionais para SDN**. Tese (Doutorado) — Universidade de São Paulo, 2018.
- BHARDWAJ, Shanu; PANDA, Surya Narayan. Performance evaluation using ryu sdn controller in software-defined networking environment. **Wireless Personal Communications**, Springer, v. 122, n. 1, p. 701–723, 2022.
- BLANDFORD, Andrea. **Working from Home Increases Cyberattack Frequency by 238%** Disponível em: <https://www.alliancevirtualoffices.com/virtual-office-blog/remote-work-statistics-costs/>. Acessado em 01 fev. 2024.
- CHANDROTH, Jisi; ROH, Byeong-Hee; ALI, Jehad. Performance analysis of python based sdn controllers over real internet topology. In: IEEE. **2022 Thirteenth International Conference on Ubiquitous and Future Networks (ICUFN)**. [S.l.], 2022. p. 283–288.
- GILMAN, Evan; BARTH, Doug. **Zero trust networks**. [S.l.]: O'Reilly Media, Incorporated, 2017.
- GOMES, Lucas de Carvalho et al. **Redes Definidas por Software - UFRJ**. 2015. Disponível em: https://www.gta.ufrj.br/ensino/eel879/trabalhos_vf_2015_2/SDN/architecture.html. Acessado em 05 fev. 2024.

- GUEDES, Dorgival et al. **Redes Definidas por Software: uma abordagem sistêmica para o desenvolvimento de pesquisas em Redes de Computadores**. 2014.
- GUO, Xian et al. An intelligent zero trust secure framework for software defined networking. **PeerJ Computer Science**, PeerJ Inc., v. 9, p. e1674, 2023.
- HOQUE, Nazrul; BHATTACHARYYA, Dhruva K; KALITA, Jugal K. Botnet in ddos attacks: trends and challenges. **IEEE Communications Surveys & Tutorials**, IEEE, v. 17, n. 4, p. 2242–2270, 2015.
- JANGLA, Kinnary; JANGLA, Kinnary. Docker compose. **Accelerating Development Velocity Using Docker: Docker Across Microservices**, Springer, p. 77–98, 2018.
- JOHL, Jason. **What Are APIs, and How Does an API Work?** 2022. Disponível em: <https://blogs.mulesoft.com/learn-apis/api-led-connectivity/what-are-apis-how-do-apis-work/>. Acessado em 06 fev. 2024.
- KARAKUS, Murat; DURRESI, Arjan. A survey: Control plane scalability issues and approaches in software-defined networking (sdn). **Elsevier**, 2016.
- KBV. **Global SDN Orchestration Market Size, Share & Industry Trends Analysis Report**. 2022. Disponível em: <https://www.kbvresearch.com/sdn-orchestration-market/>. Acessado em 15 fev. 2024.
- KHONJI, Mahmoud; IRAQI, Youssef; JONES, Andrew. Phishing detection: a literature survey. **IEEE Communications Surveys & Tutorials**, IEEE, v. 15, n. 4, p. 2091–2121, 2013.
- KUROSE, James F; ROSS, Keith W. **Redes de computadores, uma abordagem top-down**. [S.l.]: Pearson, 2013.
- LERNER, Andrew; WATTS, John. **Hype Cycle for Zero Trust Networking**. 2023. Disponível em: <https://www.fortinet.com/br/solutions/gartner-hype-cycle-zero-trust>. Acessado em 01 fev. 2024.
- MAJEWSKI, Mike. **Software-Defined Networking: Where It Came From and Where It's Going**. 2023. Disponível em: <https://www.devprojournal.com/technology-trends/software-defined-networking-where-it-came-from-and-where-its-going/>. Acessado em 01 fev. 2024.
- NADEAU, Thomas D; GRAY, Ken. **SDN: Software Defined Networks: An authoritative review of network programmability technologies**. [S.l.]: O'Reilly Media, Inc., 2013.
- O'KANE, Philip; SEZER, Sakir; CARLIN, Domhnall. Evolution of ransomware. **Iet Networks**, Wiley Online Library, v. 7, n. 5, p. 321–327, 2018.
- OSANAIYE, Opeyemi; CHOO, Kim-Kwang Raymond; DLODLO, Mqhele. Distributed denial of service (ddos) resilience in cloud: Review and conceptual cloud ddos mitigation framework. **Journal of Network and Computer Applications**, v. 67, p. 147–165, 2016. ISSN 1084-8045. Disponível em: <https://www.sciencedirect.com/science/article/pii/S1084804516000023>.

PATIL, Vinay Tila; DEORE, Shailesh Shivaji. Strategies and horizons in ddos attack detection: An analytical and predictive study. **Available at SSRN 4727005**, 2024.

PINGIDENTITY. **Zero Trust Security Model**. 2021. Disponível em: <https://www.pingidentity.com/en/resources/identity-fundamentals/zero-trust-security.html>. Acessado em 31 jan. 2024.

PRAJAPATI, Arpita; SAKADASARIYA, Achyut; PATEL, Jitisha. Software defined network: Future of networking. In: IEEE. **2018 2nd international conference on inventive systems and control (ICISC)**. [S.l.], 2018. p. 1351–1354.

RUAMBO, Francis A et al. Securing sdn/nfv-enabled campus networks with software-defined perimeter-based zero-trust architecture. **Bin, Securing Sdn/Nfv-Enabled Campus Networks with Software-Defined Perimeter-Based Zero-Trust Architecture**, 2023.

SAXENA, Smruti; SONI, Hemant Kumar. Strategies for ransomware removal and prevention. In: IEEE. **2018 Fourth International Conference on Advances in Electrical, Electronics, Information, Communication and Bio-Informatics (AEEICB)**. [S.l.], 2018. p. 1–4.

SIDDIQUI, Tamanna; SIDDIQUI, Shadab Alam; KHAN, Najeeb Ahmad. Comprehensive analysis of container technology. In: IEEE. **2019 4th International Conference on Information Systems and Computer Networks (ISCON)**. [S.l.], 2019. p. 218–223.

SINGH, Vindeep; PEDDOJU, Sateesh K. Container-based microservice architecture for cloud applications. In: IEEE. **2017 International Conference on Computing, Communication and Automation (ICCCA)**. [S.l.], 2017. p. 847–852.

SRINIVASAN, Priyadharsini. Zero trust network architecture. 2023.

STEFANINI. **O que é SDN? Entenda o conceito de software defined network**. 2018. Disponível em: <https://stefanini.com/pt-br/insights/artigos/entenda-o-conceito-de-software-defined-network>. Acessado em 24 jan. 2024.

SYED, Naeem Firdous et al. Zero trust architecture (zta): A comprehensive survey. **IEEE Access**, IEEE, v. 10, p. 57143–57179, 2022.

TUNCER, Daphne et al. A northbound interface for software-based networks. In: IEEE. **2018 14th International Conference on Network and Service Management (CNSM)**. [S.l.], 2018. p. 99–107.

YAN, Qiao et al. Software-defined networking (sdn) and distributed denial of service (ddos) attacks in cloud computing environments: A survey, some research issues, and challenges. **IEEE communications surveys & tutorials**, IEEE, v. 18, n. 1, p. 602–622, 2015.

ZANASI, Claudio; RUSSO, Silvio; COLAJANNI, Michele. Flexible zero trust architecture for the cybersecurity of industrial iot infrastructures. **Ad Hoc Networks**, Elsevier, 2024.